

An Almost-Linear Approximation Scheme for Edit Distance

OpenAI

September 24, 2026

Abstract

We give a uniform randomized approximation scheme for unit-cost edit distance. For every fixed rational $\varepsilon \in (0, 1)$, it estimates the distance between arbitrary explicitly stored strings of total length N within a factor $1 + \varepsilon$ with probability at least $2/3$, in worst-case expected time $N^{1+o(1)}$ on a logarithmic-word RAM. The algorithm supports polynomially bounded integer alphabets and returns zero deterministically on equal strings.

1 Introduction

The unit-cost edit distance $\text{ED}(x, y)$ is the minimum number of single-symbol insertions, deletions and substitutions that transform a string x into a string y . All three operations have cost one. The strings are stored explicitly; write $N = |x| + |y|$. Their symbols are integer labels of polynomial magnitude in $N + 2$, stored in words of $O(\log(N + 2))$ bits. There is no bound on the alphabet size independent of N .

An approximation scheme receives a rational $\varepsilon \in (0, 1)$ and estimates the distance multiplicatively. Its guarantee must therefore include returning zero when the two strings are equal. We prove the following result.

Theorem 1.1. *There is one uniform randomized algorithm that, given x, y and a rational $\varepsilon \in (0, 1)$, returns a nonnegative integer $\hat{D}$ such that*

$$\Pr[\text{ED}(x, y) \leq \hat{D} \leq (1 + \varepsilon) \text{ED}(x, y)] \geq \frac{2}{3}.$$

For every fixed rational ε , its worst-case expected running time on strings of total length N is $N^{1+o(1)}$ as $N \rightarrow \infty$ on the logarithmic-word RAM. The expectation is over internal randomness only, and the bound includes input processing, preprocessing, numerical work, random sampling and all data-structure operations. Equal strings return zero deterministically.

The program is uniform in the accuracy input. The asymptotic assertion is pointwise in each fixed rational accuracy: its constants and the threshold at which the bound applies may depend on ε . We do not assert polynomial dependence on $1/\varepsilon$ or a joint bound when the accuracy varies with N . The output is a distance estimate; no edit-script reconstruction is needed.

The algorithm uses exact computation below an accuracy-dependent input-length threshold. Section 2 gives the finite input convention and fallback rules; their thresholds are very large, so the result is an asymptotic approximation scheme.

1.1 Context and approach

Edit distance originates in the study of errors in transmitted strings, including Levenshtein’s binary codes for insertions, deletions and bit flips [19]. The dynamic program of Wagner and Fischer [29] computes the minimum cost of insertions, deletions and substitutions in quadratic time. Masek and Paterson [22] applied block tabulation to obtain a logarithmic speedup for fixed alphabets and suitable fixed costs. Distance-sensitive exact algorithms offer another route when few edits are needed: Ukkonen [28] developed bounded-distance methods, and Landau and Vishkin [17, 18] combined furthest-reaching states with fast matching queries. Myers’s related difference algorithm [23] treats insertion–deletion scripts. For unrestricted exact computation, Backurs and Indyk [5] showed that a strongly subquadratic deterministic algorithm would refute the Strong Exponential Time Hypothesis. That lower bound does not rule out fixed-factor approximation.

Approximation algorithms progressively improved the accuracy available with substantially less work. In the following historical comparisons, write n for a common upper bound on the string lengths. Batu and coauthors [7] gave sublinear tests for fixed alphabets that distinguish sufficiently small distance from linear distance. For general pairs, Bar-Yossef, Jayram, Krauthgamer and Kumar [6] obtained an $n^{3/7}$ approximation in quasi-linear time; Batu, Ergün and Sahinalp [8] improved the factor to $n^{1/3+o(1)}$. For binary strings, Ostrovsky and Rabani [25] constructed a low-distortion embedding into ℓ_1 , and Andoni and Onak [2] obtained an efficiently computable $2^{\tilde{O}(\sqrt{\log n})}$ approximation in $n^{1+o(1)}$ time. Andoni, Krauthgamer and Onak [3] then achieved a $(\log n)^{O(1/\xi)}$ factor in $n^{1+\xi}$ time for each fixed $\xi > 0$, using a hierarchical alignment representation and nonuniformly allocated precisions.

The next advances reached constant factors. Chakraborty, Das, Goldenberg, Koucký and Saks [10] gave a constant-factor approximation in $\tilde{O}(n^{12/7})$ time. Brakensiek and Rubinfeld [9] and Koucký and Saks [16] independently obtained near-linear exponents for sufficiently far input pairs, with a material additive error on general pairs. Andoni and Nosatzki [1] removed that restriction: for every fixed $\xi > 0$, their algorithm gives an approximation factor depending only on ξ in $O(n^{1+\xi})$ time. The factor depends on the exponent parameter; this is not an approximation scheme with error tending to zero. For accuracy arbitrarily close to one, Mao and Rubinfeld [21, Theorem 6.1] recently obtained a randomized $(1 + \varepsilon)$ approximation in $n^2/2^{\log^{\Omega(1)} n}$ time for every fixed positive ε .

The issue addressed here is how to retain fixed $(1 + \varepsilon)$ accuracy while reducing the expected total work to $N^{1+o(1)}$. Recursive interval decompositions, economical evaluation of local distances, and accounting along an optimal alignment have substantial precedent in the algorithms above. In our refinement, unrestricted recursive evaluation of child distances would consume the available time even if the resulting numerical error were small. We therefore need both a narrow family that contains a suitable comparison alignment and a bound on the number of child distances actually evaluated.

For the initial estimates we adapt the shift tree and precision sampling of Andoni, Krauthgamer and Onak [3, 4]. The refinement is related to Mao and Rubinfeld’s use of multiscale deviation to constrain alignment paths before sampling [21]. Here we measure the alignment’s changes in relative position using preliminary cost estimates as weights. A telescoping bound on their variation controls the total cost of places where the prediction fails. An entropy-regularized prediction rule, within the framework of Rakhlin and Sridharan [26], then controls the accumulated error where the predictions can be used. We implement its optimization by a finite Frank–Wolfe procedure [12, 15]. The specialized geometric, probabilistic and numerical guarantees are proved in this paper.

Two related comparisons concern different requirements on the output or the edit costs. A single ℓ_1 embedding of all binary strings up to a length bound must incur growing distortion [24, Theorem 1.1]. Our algorithm estimates the distance of the supplied pair and does not construct such

a common embedding. Recent algorithms for metric-weighted edit costs obtain $(3+\varepsilon)$ approximation in strongly subquadratic time [11, 20]; their cost model is broader than the unit-cost model treated here.

1.2 Proof overview

Recursively partition the first string into a tree of intervals, ending at intervals of length at most one. An interval state consists of a source interval I and a target interval in the second string; its true cost is the edit distance between the two substrings. An action at a nonleaf assigns one target interval to each source child. We charge the sum of the child estimates together with penalties for gaps or overlaps between successive target endpoints and at the two outer endpoints. These penalties let us compare nearby actions without requiring their target intervals to join exactly.

Estimates that can be requested adaptively. The initial seed is an integer table whose entries give coarse upper approximations to all interval-state costs. Although the algorithm evaluates only requested entries, the accuracy event covers the entire domain: later queries may depend on earlier random answers. Each initial entry uses a private random stream, so its expected evaluation cost can also be bounded conditional on the request that first exposes it. Accuracy and expected work thus use different probability statements. Successive refinement passes improve the seed approximation factor.

Predicting where an alignment goes. One refinement starts from such a seed. It first propagates estimates through a wide family of actions and averages the resulting preliminary tables to obtain cost proxies. At a source cut, the alignment’s shift is the target position minus the source position. The total shift change divided by the predicted total cost suggests how this shift should evolve as prefix cost accumulates. Requiring agreement with that prediction restricts the possible target positions at each cut to a narrow band. The algorithm computes these bands from the cost proxies alone.

To analyze them, fix optimal alignments and recursively round their child endpoints. This gives a comparison tree that the algorithm never computes. A cost-weighted variance measures how far the comparison shifts depart from the predicted evolution. Its sum telescopes, with additional charges for disagreements among the cost proxies. Consequently the total cost of nodes with large discrepancies is controlled. At the other nodes, either the substring-length difference already accounts for almost all predicted cost, giving a direct error bound, or the computed band contains the comparison action.

From a band to a better seed. Within a band, an entropy-regularized rule predicts an action using earlier child values. Shared samples also compare rounded actions across nearby representatives. The resulting estimates decrease with the refinement index. After conditioning on complete child tables, the parent samples retain their prescribed laws, and an online inequality bounds cumulative excess over the comparison action. Some intermediate estimates can be smaller than true cost. The analysis therefore clips them from below at true cost, so an underestimate at one node cannot cancel an error elsewhere. Summing over the comparison tree gives a small expected positive error at any fixed queried state, together with a deterministic multiplicative lower bound. Independent repetitions, rescaling and medians turn these two bounds into a simultaneous one-sided seed with a smaller approximation factor. Repeating this improvement and making one final refinement gives the desired answer.

Paying for recursive queries. A wide search or an optimization step may inspect many actions, but it reuses previously gathered child values. Only shared group samples, selected rounded actions and actual online draws require child values at the current refinement index. Common rounded grids limit the number of these calls to slightly more than the tree’s branching factor. Along a dependency path, a call either descends the tree at the same index or decreases an index or pass. Counting these two kinds of steps bounds the total expected work, including rational arithmetic and exact sampling, by $N^{1+o(1)}$.

Section 2 fixes the computational model, geometry and parameters. Sections 3–5 construct the tables. Sections 6 and 7 prove the geometric and probabilistic estimates, and Section 8 assembles the accuracy guarantee. Sections 9 and 10 implement every operation and complete the running-time proof.

2 Preliminaries

2.1 Inputs and basic parameters

Write $N = |x| + |y|$. The strings are stored explicitly, and their symbols are integers of polynomial magnitude in $N + 2$. We count operations on a RAM with words of $O(\log(N + 2))$ bits. Larger integers and rational numbers are stored in several words, and their arithmetic is charged.

The accuracy input is a rational $\varepsilon \in (0, 1)$ on a separate read-only input. Its positive numerator and denominator are in lowest terms and have a fixed canonical binary encoding. Encode each integer by its binary length in unary, a delimiter, and its binary digits. Concatenate the two integer encodings. Reading this input and performing arithmetic on its integers both contribute to the work.

Let H be the least power of two at least $\lceil \log_2(N + 2) \rceil$. Let ℓ be the least power of two at least $\max\{1, \log_2 H\}$. Thus $H = \Theta(\log(N + 2))$ and $\ell = \Theta(\log H)$ as N grows. We use $\ln$ for the natural logarithm. The large-input branch has $\ell \geq 2^{1000}$ and $\ell\varepsilon \geq 10$. The algorithm first handles equality, empty strings, and $|x| \leq 1$ exactly in $O(N + 1)$ string work. On other inputs, it inspects at most $H^2 + O(1)$ bits of the accuracy encoding. If the encoding is incomplete, it uses exact dynamic programming without reading or addressing the suffix. Otherwise its numerator and denominator have $O(H^2)$ bits, and integer arithmetic tests $\ell\varepsilon \geq 10$ in polynomial work in H . The inspected addresses have $O(\log(N + 2))$ bits.

If either numerical branch condition fails, the algorithm also uses exact dynamic programming. This takes $O((N + 1)^2)$ word operations and two rows of storage. For each fixed rational encoding, inspection eventually reads the complete input and both numerical tests eventually pass. The exact branches occupy a bounded range of N for each fixed accuracy.

For the bounded parser, use the fixed allowance $c_{\text{enc}} = 4$. It returns INCOMPLETE before reading beyond its bit budget.

Algorithm 1 Inspecting only a bounded accuracy prefix

```
1: function READENCODEDINTEGER( $e, p, L_{\text{bits}}$ )
2:    $h \leftarrow 0$ 
3:   repeat
4:     if  $p = L_{\text{bits}}$  then
5:       return INCOMPLETE
6:     end if
7:      $b \leftarrow \text{bit } e[p + 1]; p \leftarrow p + 1$ 
8:     if  $b = 1$  then
9:        $h \leftarrow h + 1$ 
10:    end if
11:  until  $b = 0$ 
12:  if  $p + h > L_{\text{bits}}$  then
13:    return INCOMPLETE
14:  end if
15:   $v \leftarrow 0$ 
16:  for  $j = 1, \dots, h$  do
17:     $b \leftarrow \text{bit } e[p + 1]; p \leftarrow p + 1; v \leftarrow 2v + b$ 
18:  end for
19:  return  $(v, p)$ 
20: end function
21: function INSPECTACCURACY( $\varepsilon, L_{\text{bits}}$ )
22:    $e_{\text{num}} \leftarrow \text{READENCODEDINTEGER}(\varepsilon, 0, L_{\text{bits}})$ 
23:   if  $e_{\text{num}} = \text{INCOMPLETE}$  then
24:     return INCOMPLETE
25:   end if
26:   Write  $e_{\text{num}} = (v_{\text{num}}, p)$ 
27:    $e_{\text{den}} \leftarrow \text{READENCODEDINTEGER}(\varepsilon, p, L_{\text{bits}})$ 
28:   if  $e_{\text{den}} = \text{INCOMPLETE}$  then
29:     return INCOMPLETE
30:   end if
31:   return  $(v_{\text{num}}, v_{\text{den}})$ , where  $e_{\text{den}} = (v_{\text{den}}, p')$ 
32: end function
```

Algorithm 2 Exact dynamic programming

```
1: function EXACTDP( $x, y$ )
2:    $D[j] \leftarrow j$  for  $0 \leq j \leq |y|$ 
3:   for  $i = 1, \dots, |x|$  do
4:      $E[0] \leftarrow i$ 
5:     for  $j = 1, \dots, |y|$  do
6:        $E[j] \leftarrow \min\{D[j] + 1, E[j - 1] + 1, D[j - 1] + \mathbf{1}_{x_i \neq y_j}\}$ 
7:     end for
8:     Swap the rows  $D$  and  $E$ 
9:   end for
10:  return  $D[|y|]$ 
11: end function
```

2.2 Alignment paths and endpoint pairs

Index string boundaries from zero. An alignment path has steps $(1, 0)$, $(0, 1)$, and $(1, 1)$. The first two steps have cost one. A diagonal step has cost zero when its two symbols agree and cost one otherwise. Every alignment specifies an edit sequence of the same cost. Conversely, the original

symbols that survive an edit sequence retain their order. Delete the other original symbols, insert the remaining output symbols, and substitute each surviving symbol whose label differs. These operations give an alignment of no greater cost. Minimum alignment cost equals edit distance.

For a source interval $I = (l, u]$, let $q = (q^-, q^+)$ be a *target endpoint pair*, with $0 \leq q^- \leq q^+ \leq |y|$. The pair (I, q) is an interval state. When I is fixed, we also call q a state. Define

$$c(I, q) = \text{ED}(x_{(l, u]}, y_{(q^-, q^+]}), \quad z(I, q) = (q^+ - q^-) - (u - l).$$

An endpoint's *shift* is its target index minus its source index. The quantity $z(I, q)$ is the difference between the final and initial shifts. For two endpoint pairs, write $|q - r|_1 = |q^- - r^-| + |q^+ - r^+|$.

Lemma 2.1. *The following statements hold.*

1. *For every interval state, $c(I, q) \geq |z(I, q)|$. Both $c(I, \cdot)$ and $|z(I, \cdot)|$ are 1-Lipschitz in the endpoint metric.*
2. *Suppose that $I_1, \dots, I_m$ partition I in order, and let $\alpha = (s_1, \dots, s_m)$ be any list of valid child endpoint pairs. Set*

$$\text{conn}_r(\alpha) = |s_1^- - r^-| + \sum_{i=1}^{m-1} |s_{i+1}^- - s_i^+| + |s_m^+ - r^+|.$$

Then

$$\begin{aligned} c(I, r) &\leq \sum_i c(I_i, s_i) + \text{conn}_r(\alpha), \\ |z(I, r)| &\leq \sum_i |z(I_i, s_i)| + \text{conn}_r(\alpha). \end{aligned}$$

These two inequalities also hold for finite target intervals in an extension of y beyond its stored endpoints.

Proof. Only insertions and deletions change length, which proves $c \geq |z|$. Moving the target endpoints from q to r costs at most $|q - r|_1$. For overlapping substrings, insert or delete the endpoint portions. For disjoint substrings, delete the first substring and insert the second. The triangle inequality proves the Lipschitz bound for c . The formula for z proves the other Lipschitz bound.

For the connection inequality, first edit each source child to its specified target substring and concatenate the results. Consecutive child targets may have a gap or an overlap, so this concatenation need not be the parent's target. We show that correcting it costs at most $\text{conn}_r(\alpha)$.

Consider the walk on the target boundary line that starts at r^- , jumps to s_1^- , traverses each child target forward, makes the connecting jumps, and ends at r^+ . Expand each jump into unit steps. Select one forward crossing of each unit interval between r^- and r^+ , in that order. The successive first arrivals at these boundaries give such a selection. Its letters form exactly the parent's target substring.

Delete letters on unselected child crossings and insert letters on selected connecting crossings. Net displacement shows that the number of unselected forward crossings equals the number of backward crossings. Every backward crossing belongs to a connecting jump. It can pay for one deletion. A selected forward connecting crossing pays for its own insertion. The total correction cost is at most the sum of the forward and backward jump lengths, which is $\text{conn}_r(\alpha)$. This proves the cost inequality. The walk uses only finitely many target positions, so the argument also applies to an extension of y .

Finally, sum the signed shift changes of the child intervals and the connecting jumps. Taking absolute values proves the inequality for $|z|$. $\square$

Figure 1 shows the connecting jumps for child targets with both a gap and an overlap.

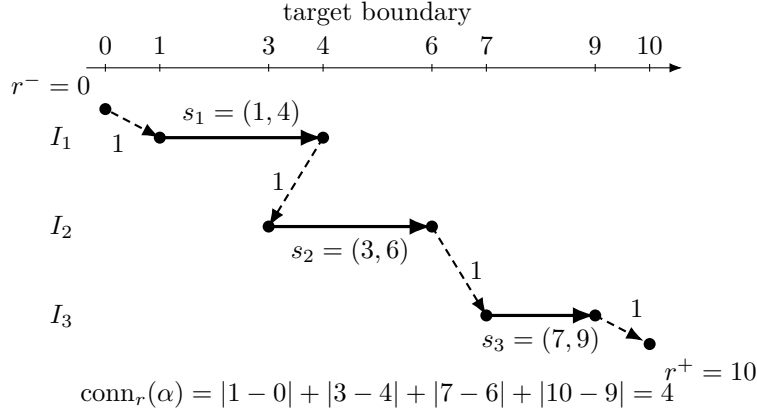


Figure 1: Connection charges for three consecutive source children. Solid arrows show their prescribed target intervals; dashed arrows join these intervals in child order, with the outside endpoints $r = (0, 10)$. The jump from 4 to 3 points backward, so this walk is not itself an alignment path. Lemma 2.1 gives $c(I, r) \leq \sum_{i=1}^3 c(I_i, s_i) + 4$. Only the horizontal target-coordinate displacement of a dashed arrow is charged; the vertical separation of rows is for display.

2.3 The source interval tree and exact leaf queries

In the large-input branch, set $M = 2^{\lfloor (\log_2 H)/20 \rfloor}$. Partition x into a complete M -ary tree of height

$$J = \lceil \log_M \max\{1, |x|\} \rceil.$$

At a node $I = (l, u]$, use cuts $l_i = l + \lfloor i(u - l)/M \rfloor$ for $0 \leq i \leq M$. The children are $I_i = (l_{i-1}, l_i]$. Empty children are allowed. Node labels distinguish repeated intervals, and nodes can be generated when requested.

Lemma 2.2. *In the large-input branch, the source tree satisfies*

$$J \leq 100H/\ell, \quad M^J \leq M \max\{1, |x|\}.$$

A node at depth j has width at most $\lceil |x|/M^j \rceil$. Every leaf has width at most one.

Proof. The floor cuts give the width bound at each level. Also $\log_2 M \geq (\log_2 H)/20 - 1$, $\log_2 H \geq \ell/2$, and $\log_2 |x| \leq H$. These inequalities bound the ceiling defining J by $100H/\ell$. The same ceiling gives $M^J \leq M \max\{1, |x|\}$. At depth J , the width bound is at most one. $\square$

For a width-zero leaf, $c(I, q)$ is the target length. For a leaf containing one symbol, the cost is one if the target is empty. Otherwise, it is the target length minus the indicator that the source symbol occurs in the target interval. Sort the pairs (symbol, position) of y lexicographically. Binary search then answers an occurrence query in $O(\log(N + 2))$ time. Preprocessing costs $O(N \log(N + 2))$ work and uses linear storage. Symbol comparisons take a bounded number of word operations. A fresh padding symbol can be stored as a tag and a label.

All tables have this fixed source tree as their node domain. There are at most $2M^J$ nodes and at most $(|y| + 1)^2$ valid endpoint pairs per node. This polynomial domain permits simultaneous probability guarantees, although the algorithm evaluates only the entries it requests.

The pseudocode uses the shared input strings and parameters. The call $\text{READ}(R, I, q)$ requests one entry of a table descriptor R , and $\text{GET}(k)$ requests a table entry or auxiliary object by its full key. A first request runs the defining procedure and stores its result; subsequent requests return the stored value. Each random object has an independent private source ω . The procedure $\text{UNIFORMINT}(m, \omega)$ draws a uniform integer from $\{1, \dots, m\}$ using that source.

Node arguments include their labels and remaining heights. For $I = (l, u]$, $\text{CHILDREN}(I, b)$ uses the floor-cut rule above with branching factor b . The cuts l_i can be recovered from I and b . String fragments and padded strings are views with constant-time character access. Grids are generated from integer index ranges, and action lists are finite Cartesian products. We use lexicographic order to break ties. Searches gather distinct recursive inputs before repeatedly scoring actions. An empty minimum is $+\infty$.

Algorithm 3 Implicit children and fair integer samples

```

1: function CHILDREN( $I, b$ )
2:   Let  $I = (l, u]$  have node label  $\nu$  and remaining height  $h > 0$ 
3:    $l_i \leftarrow l + \lfloor i(u - l)/b \rfloor$  for  $0 \leq i \leq b$ 
4:   return the nodes  $((l_{i-1}, l_i], (\nu, i), h - 1)$  for  $1 \leq i \leq b$ 
5: end function
6: function UNIFORMINT( $m, \omega$ )
7:    $h \leftarrow \min\{j \geq 0 : 2^j \geq m\}$ 
8:   repeat
9:     Draw  $h$  fair bits from  $\omega$ , interpreted as  $R \in \{0, \dots, 2^h - 1\}$ 
10:  until  $R < m$ 
11:  return  $R + 1$ 
12: end function

```

Algorithm 4 Exact leaf costs

```

1: function LEAFCOST( $I, q$ )
2:    $n_q \leftarrow q^+ - q^-$ 
3:   if  $|I| = 0$  then
4:     return  $n_q$ 
5:   end if
6:   if  $n_q = 0$  then
7:     return 1
8:   end if
9:   Use the occurrence index to test whether  $x_{l+1}$  occurs in  $y_{(q^-, q^+]}$ 
10:  return  $n_q - \mathbf{1}_{\text{an occurrence exists}}$ 
11: end function

```

3 Simultaneous coarse estimates

The refinement will need an integer estimate at every interval state, including states chosen adaptively by later computations. We construct these estimates as a random table, while evaluating only the entries that are requested. The construction builds on tree and precision-sampling ideas of Andoni, Krauthgamer, and Onak [3]; the complete bounds needed here are proved below.

The coarse construction uses

$$p_* = \left\lfloor (\log_2 \ell)^{1/4} \right\rfloor, \quad B = H^{p_*}, \quad A_{\text{init}} = H^{p_*+4}.$$

Here B is the branching factor of the estimator's tree, and A_{init} is its final approximation factor. This tree is separate from the source tree, whose branching factor is M . The integer p_* is the largest integer whose fourth power is at most $\log_2 \ell$. Thus computing these parameters requires only integer tests.

Let U_0 be the descriptor of the initial seed table. Each requested entry uses its own private source, shared by all guesses and sampled allowances in that entry. The completed integer is stored under (U_0, I, q) .

3.1 A deterministic shift tree

The hierarchical representation below is an adaptation of the E-distance of Andoni, Krauthgamer and Onak [3]. We prove its precise bounds here, including the treatment of unequal lengths, empty intervals and bounded integer shifts.

Fix one state and write $n_x = |I|$, $n_y = q^+ - q^-$, and $c = c(I, q)$. If $n_y > 2n_x$, return n_y : the bounds $n_y - n_x \leq c \leq n_y$ give the required factor two. Otherwise test equality directly and return zero when the strings agree. These steps cost $O(1 + n_x)$, since in the branch requiring character comparisons $n_y \leq 2n_x$. They include the case $n_x = 0$.

For the remaining case, pad the shorter string at its right end with a fresh tagged symbol to obtain two strings of the same length $n \leq 2n_x$. Denote their distance by d . The length gap is at most c , so $d \leq 2c$. Conversely, remove padding from any padded alignment. A diagonal joining padding to a real character has cost one and can be replaced by an insertion or deletion of the same cost; other operations consuming padding can be removed. Hence

$$1 \leq c \leq d \leq 2c, \quad 1 \leq n \leq N, \quad d \leq n.$$

The last bound follows by substituting characters position by position.

For each dyadic guess

$$k \in \{1, 2, 4, \dots, 2^{\lceil \log_2 n \rceil}\},$$

split the first padded string in a complete balanced B -ary tree of depth $D_0 = \lceil \log_B n \rceil$. Its leaves have width zero or one. Allowed shifts are the integers in $[-k, k]$. At a width-one leaf w starting at position j , let $T_w(v)$ be the mismatch indicator between its character and position $j + 1 + v$ of the second padded string. A position outside the latter string has a virtual symbol that always mismatches. At a width-zero leaf put $T_w(v) = 0$.

Definition 3.1. At an internal node define

$$T_w(v) = \sum_{i=1}^B \min_{v' \in [-k, k] \cap \mathbb{Z}} \{T_{w_i}(v') + 2|v - v'|\}.$$

Constant shifts show that $0 \leq T_w(v) \leq |w|$, where $|w|$ denotes the node's width.

The virtual symbols are chosen outside the alphabets of both padded strings. Thus every leaf value is an edit cost against a finite interval of an extended target string, to which Lemma 2.1 applies.

Lemma 3.2. *At the root o ,*

$$d \leq T_o(0), \quad T_o(0) \leq (1 + 2BD_0)d \quad \text{if } k \geq d.$$

Proof. The recurrence minimizes the sum of leaf mismatches and twice the total absolute shift difference along tree edges, with root shift zero. For any assignment, concatenate the shifted leaf target intervals. By traversing the tree in depth-first order, the sum of jumps between consecutive leaf intervals, including the connections to the outer endpoints, is at most twice the sum of edge shift differences. The connection inequality from Lemma 2.1, including its virtual-position version, bounds d by the assignment's cost. This proves the first assertion.

For the second, fix an optimal padded alignment and choose ordered points (j, b_j) at every source cut, with $b_0 = 0$ and $b_n = n$. Assign shift $b_j - j$ to every node starting at j . Its absolute value is at most d , because only insertion and deletion steps change the shift. These assignments are allowed when $k \geq d$.

For each child, the shift difference from its parent is at most the cost of the alignment subpath between their chosen starting points, and thus at most the cost of the parent's subpath. There are B children per parent; the parent subpaths at each depth partition the alignment. Edge differences sum to at most Bd at each of the D_0 depths. A mismatching width-one leaf has positive cost in its corresponding subpath: a zero-cost subpath would be a matching diagonal. Leaf mismatches sum to at most d , and width-zero leaves contribute nothing. The assignment has total cost at most $(1 + 2BD_0)d$. $\square$

3.2 Pruning and precision sampling

Nonuniform additive precision is the sampling principle developed by Andoni, Krauthgamer and Onak [3] and refined in [4], with earlier multiscale sampling antecedents in Indyk and Woodruff [14]. The reconstruction arguments of Andoni, Krauthgamer and Onak already allow estimates to depend on the sampled precisions. Here we use a finite threshold distribution and prove an event covering all potential interval states, together with the conditional cost of a first request.

Computing all of Definition 3.1 would be too expensive. The guess k will also serve as an additive allowance at the root: on one event of high probability, we will obtain an estimate $\tilde{T}_o$ between $T_o/2 - k$ and $2T_o + k$ simultaneously for every entry and guess. Then $2(\tilde{T}_o(0) + k)$ is an upper bound on d for every guess, and the guess $d \leq k < 2d$ gives an $O(1 + BD_0)$ approximation. We can replace a node by zero whenever its entire cost fits within its additive allowance. Larger guesses permit more pruning; their larger allowances are paid back in the returned estimate.

Definition 3.3. Set

$$Q_0 = H^4, \quad \lambda = \frac{1}{64H}, \quad a_o = k.$$

At a node with allowance a_w , independently for its children draw R_i uniformly from $\{1, \dots, B\}$ and set

$$t_i = \frac{a_w R_i}{BQ_0}, \quad a_{w_i} = \lambda t_i.$$

All these random choices can be defined in advance, even below nodes that will be pruned. The implementation exposes only those it needs.

If $|w| \leq a_w$, use the identically zero estimate $\tilde{T}_w$. At any other leaf use its exact array. At an unpruned internal node first compute

$$\hat{s}_i(v) = \min_{v' \in [-k, k] \cap \mathbb{Z}} \{\tilde{T}_{w_i}(v') + 2|v - v'|\},$$

and then put

$$\tilde{T}_w(v) = \sum_{i=1}^B \mathbf{1}_{\hat{s}_i(v) \geq t_i} \max\{\hat{s}_i(v), a_w/Q_0\}.$$

The descendants used to compute $\widehat{s}_i(v)$ depend on t_i , so these child estimates cannot be treated as fixed in a concentration bound. The proof below first compares their sampled contributions with two lists formed from true child costs. It then concentrates those lists without assuming that any child estimate is accurate.

For an array E on $[-k, k] \cap \mathbb{Z}$, its penalty transform is computed by a left sweep

$$L(v) = \min\{E(v), L(v-1) + 2\}$$

with the natural initial value, and the symmetric right sweep. Their pointwise minimum is

$$\min_{v'} \{E(v') + 2|v - v'|\}.$$

Both sweeps take $O(k)$ operations. An identically zero child array, whose transform is also zero, is represented by a marker without allocation.

Algorithm 5 The shift penalty transform

```

1: function PENALTYTRANSFORM( $E, k$ )
2:   if  $E$  is the zero marker then
3:     return the zero marker
4:   end if
5:    $L[-k] \leftarrow E[-k]; R[k] \leftarrow E[k]$ 
6:   for  $v = -k + 1, \dots, k$  do
7:      $L[v] \leftarrow \min\{E[v], L[v-1] + 2\}$ 
8:   end for
9:   for  $v = k - 1, \dots, -k$  in decreasing order do
10:     $R[v] \leftarrow \min\{E[v], R[v+1] + 2\}$ 
11:   end for
12:   return the array  $\min\{L[v], R[v]\}$  on  $[-k, k] \cap \mathbb{Z}$ 
13: end function

```

Algorithm 6 Pruned shift-tree estimate

Input: Equal-length padded views X, V and the initial entry's source ω

```

1: function COARSENODE( $w, k, a_w, X, V, \omega$ )
2:   if  $|w| \leq a_w$  then
3:     return the zero marker
4:   end if
5:   if  $w$  has remaining height zero then
6:     Let  $j$  be its starting boundary
7:     return  $(\mathbf{1}_{X_{j+1} \neq V_{j+1+v}})_{v=-k}^k$ , with out-of-range positions always mismatching
8:   end if
9:    $(w_i)_{i=1}^B \leftarrow \text{CHILDREN}(w, B)$ 
10:  Allocate  $E[v] \leftarrow 0$  for  $v = -k, \dots, k$ 
11:  for  $i = 1, \dots, B$  do
12:     $R_i \leftarrow \text{UNIFORMINT}(B, \omega); t_i \leftarrow a_w R_i / (BQ_0)$ 
13:     $E_i \leftarrow \text{COARSENODE}(w_i, k, \lambda t_i, X, V, \omega)$ 
14:     $\widehat{s}_i \leftarrow \text{PENALTYTRANSFORM}(E_i, k)$ 
15:    for  $v = -k, \dots, k$  do
16:       $E[v] \leftarrow E[v] + \mathbf{1}_{\widehat{s}_i(v) \geq t_i} \max\{\widehat{s}_i(v), a_w/Q_0\}$ 
17:    end for
18:    Release  $E_i$  and  $\widehat{s}_i$ 
19:  end for
20:  return  $E$ 
21: end function

```

Lemma 3.4. *Except on an event of probability less than 0.01, simultaneously for all initial entries, all guesses, all their nodes, and all shifts,*

$$T_w(v)/f_w - a_w \leq \tilde{T}_w(v) \leq f_w T_w(v) + a_w, \quad f_w = \left(\frac{1+\lambda}{1-\lambda}\right)^{\text{height}(w)} \leq 2.$$

Proof. Fix an internal node and a shift, and write s_i for the true child penalty minimum in Definition 3.1. Let f' be the factor in the accuracy inequalities at its children's structural height. Both s_i and f' are independent of all thresholds. Suppose for the moment that the child accuracy inequalities hold with this factor. Adding a nonnegative penalty preserves the lower bound after division by f' and the upper bound after multiplication by f' . Taking minima gives

$$s_i/f' - \lambda t_i \leq \hat{s}_i(v) \leq f' s_i + \lambda t_i.$$

Let $u = a_w/Q_0$ and define

$$\phi_i(h) = \mathbf{1}_{h \geq t_i} \max\{h, u\}, \quad h_i^- = \frac{s_i}{f'(1+\lambda)}, \quad h_i^+ = \frac{f' s_i}{1-\lambda}.$$

If $h_i^- \geq t_i$, then $\hat{s}_i(v) \geq (1+\lambda)h_i^- - \lambda t_i \geq h_i^-$. If $\hat{s}_i(v) \geq t_i$, then $\hat{s}_i(v) \leq f' s_i + \lambda \hat{s}_i(v)$, and consequently $\hat{s}_i(v) \leq h_i^+$. These implications prove the pathwise sandwich

$$\phi_i(h_i^-) \leq \phi_i(\hat{s}_i(v)) \leq \phi_i(h_i^+).$$

We now establish concentration independently of the supposition about child accuracy. Conditional only on ancestor thresholds, u is fixed and $t_i = uR_i/B$ are independent. For either of the two fixed argument lists $h_i = h_i^\pm$, if $0 \leq h_i < u$ then

$$\phi_i(h_i) = u \text{ Bernoulli}\left(\frac{\lfloor Bh_i/u \rfloor}{B}\right);$$

if $h_i \geq u$, it equals h_i deterministically. Thus

$$0 \leq h_i - \mathbb{E}\phi_i(h_i) \leq u/B, \quad \text{Var}(\phi_i(h_i)) \leq u h_i, \quad |\phi_i(h_i) - \mathbb{E}\phi_i(h_i)| \leq u.$$

Write $V = \sum_i h_i$ and $Z = \sum_i \phi_i(h_i)$. The total bias is at most u . Bernstein's inequality [27] at deviation $z = \lambda V + a_w/2$ gives

$$\Pr(|Z - \mathbb{E}Z| > z) \leq 2 \exp\left(-\frac{z^2}{2uV + (2/3)uz}\right) \leq 2 \exp(-\lambda Q_0/12).$$

For the last inequality, if $V \geq a_w$, the denominator is at most $3uV$ and $z^2 \geq \lambda a_w V$; if $V \leq a_w$, the denominator is at most $3ua_w$ and $z^2 \geq a_w^2/4$. Since $u \leq a_w/2$, adding the bias yields

$$|Z - V| \leq \lambda V + a_w$$

outside the indicated event.

These conditional bounds are uniform in the ancestor thresholds, so they also hold unconditionally. Take their intersection over both argument lists and all fixed node and shift labels. On that intersection, induction from the leaves combines the pathwise sandwich and the concentration bound. The lower and upper bounds become

$$\frac{1-\lambda}{f'(1+\lambda)} T_w(v) - a_w \leq \tilde{T}_w(v) \leq \frac{f'(1+\lambda)}{1-\lambda} T_w(v) + a_w,$$

which is exactly the desired factor update. A pruned node also satisfies the inequalities because $T_w(v) \leq |w| \leq a_w$; exact leaves do so directly. The factor satisfies $f_w \leq \exp(2/63) < 2$, since $D_0 \leq H$.

The event count includes the entire domain, including entries that are never requested. There are $O(M(N+1)^3)$ labeled main-tree states, $O(H)$ guesses per entry, $O(BN)$ nodes per coarse tree, and $O(N)$ shifts per guess. The count of the two lists is at most

$$CMBH(N+1)^5 \leq \exp(20H)$$

for an absolute constant C under the stated cutoff. Here one may use $B, M, H \leq N$ in this regime and $\log_2(N+2) \leq H$. The union failure is at most $2 \exp(20H - H^3/768) < 0.01$. In particular, no conditioning on successful descendants was used in the concentration calculation. $\square$

3.3 Converting the estimate to an integer seed

For the padded entry, take the ceiling of

$$\min_k 2(\tilde{T}_o(0) + k)$$

and clamp it to $[0, N]$. On the event of Lemma 3.4, every candidate is at least $T_o(0) \geq d \geq c$. A listed guess satisfies $d \leq k < 2d$, and its rounded candidate is at most

$$4(1 + 2BD_0)d + 4k + 1 \leq (25 + 16BH)c \leq BH^4c = A_{\text{init}}c.$$

Clamping preserves both bounds because $c \leq N$. The direct branches satisfy the same approximation bound.

Algorithm 7 One initial seed entry

```

1: function INITIALENTY( $I, q, \omega$ )
2:   Let  $X, V$  be views of  $x_I, y_{(q^-, q^+]}$ 
3:   if  $|V| > 2|X|$  then
4:     return  $|V|$ 
5:   end if
6:   if  $X = V$  then
7:     return 0
8:   end if
9:    $n \leftarrow \max\{|X|, |V|\}$ ;  $C_{\min} \leftarrow +\infty$ 
10:  Extend the shorter view on the right with a fresh tagged padding symbol
11:   $D_0 \leftarrow \min\{j \geq 0 : B^j \geq n\}$ 
12:  for  $k = 1, 2, 4, \dots, 2^{\lceil \log_2 n \rceil}$  do
13:     $o \leftarrow$  the implicit coarse root  $((0, n], \text{root}, D_0)$ 
14:     $E \leftarrow \text{COARSENODE}(o, k, k, X, V, \omega)$ 
15:     $C_{\min} \leftarrow \min\{C_{\min}, 2(E(0) + k)\}$ 
16:    Release  $E$ 
17:  end for
18:  return  $\min\{N, \max\{0, \lceil C_{\min} \rceil\}\}$ 
19: end function

```

3.4 The conditional cost of an adaptive request

We next prove the cost assertion, including executions outside the accuracy event. At an unpruned internal node the array work is $O(Bk)$, including visits and thresholds for all children. An unpruned leaf costs $O(k)$. A pruned node needs only its width test and marker.

Let r be a node's depth in the coarse tree. From Definition 3.3, along any fixed path,

$$a_w = \frac{k \prod R_e}{(64BH^5)^r}, \quad \mathbb{E} \left[\frac{a_{\text{parent}}}{a_w} \middle| a_{\text{parent}} \right] = 64H^5 \sum_{j=1}^B \frac{1}{j} \leq H^8.$$

It follows that $\mathbb{E}[k/a_w] \leq H^{8r}$. A node can incur array work only if $|w| > a_w$, and pointwise

$$k \mathbf{1}_{|w| > a_w} \leq |w| \frac{k}{a_w}.$$

Widths sum to n at each depth. Thus the expected array work for one guess, including child overhead charged to expanded parents, is at most

$$O(1 + Bn(D_0 + 1)H^{8D_0}).$$

This estimate uses the random allowances alone and remains valid when the estimated arrays are inaccurate.

All arithmetic and sampling can be performed with finite binary integers. The quantities in Definition 3.3 are dyadic; a common denominator for their descendants and the arrays needs $O(D_0(p_* + 5) \log H + \log H)$ bits. Minima and maxima select existing dyadic values, and sums preserve this common precision. The same-shift candidate in the penalty minimum gives

$$\tilde{T}_w(v) \leq \sum_i \tilde{T}_{w_i}(v) + Ba_w/Q_0.$$

Since $a_w \leq k$ and there are $O(Bn)$ nodes, all estimate magnitudes are bounded by $n + O(B^2nk)$. Their bit lengths, and the cost of ordinary binary arithmetic on them, fit the factor $H^{O(D_0+p_*+1)}$. Each R_i requires exactly $\log_2 B$ fair random bits. Zero arrays are never materialized.

There are $O(H)$ guesses, and

$$D_0 \leq 1 + \frac{H}{p_* \log_2 H}, \quad \log_2(H^{O(D_0+p_*+1)}) = O\left(\frac{H}{p_*} + (p_* + 1) \log H\right) = o(H).$$

Consequently the array-work bound, including the arithmetic factors, all guesses, equality tests, and padding, is $(1 + |I|)2^{o(H)}$ in expectation. Even a full coarse tree with all its shift arrays uses only polynomial space: it has $O(Bn)$ nodes, $O(n)$ entries per array, and the bit lengths just bounded.

Finally, assign a separate random stream to every initial entry, keyed by its labeled main-tree node and endpoint pair, and cache the completed answer. Its calculation does not query other initial entries. Before its first request, none of the exposed computation can depend on its unused stream: such a dependence would already have required evaluating this entry. Conditional on the entire preceding execution and the adaptive request, the stream has its original distribution. The expected cost bound applies conditionally exactly as stated; repeated requests use the cached answer and pay only lookup work. This deferred exposure realizes the full random table used in the simultaneous probability argument. The approximation and work bounds combine as follows.

Theorem 3.5. *The construction above defines a lazily evaluated random table $U^{(0)}(I, q)$ of integers in $[0, N]$ on the labeled source tree. With probability at least 0.99, its entries satisfy the bounds simultaneously at every state:*

$$c(I, q) \leq U^{(0)}(I, q) \leq A_{\text{init}} c(I, q).$$

Every zero-cost state has value zero on every execution. Conditional on any first request for an entry and the previously exposed randomness, its expected evaluation cost is at most $(1 + |I|)2^{o(H)}$, uniformly in the state. This cost bound includes arithmetic and random sampling and does not require the simultaneous seed bounds to hold.

Proof. The equality and long-target branches have the stated guarantees on every outcome. For the other entries, Lemmas 3.2 and 3.4 give the simultaneous accuracy bound proved above. The independent per-entry streams give the conditional first-request work bound. Caching an answer does not expose the unused stream of another entry. $\square$

4 Regularizing and propagating a seed

An entrywise approximation need not vary smoothly when an endpoint moves. We first replace the seed by a Lipschitz envelope. We then propagate child costs through wide actions, which permit gaps and overlaps between child targets. Averaging the propagated tables gives a stable cost proxy.

4.1 A fixed seed and refinement parameters

Definition 4.1. Let A be a power of two with $1 \leq A \leq A_{\text{init}}$. A seed is an integer table U on all labeled source nodes and valid endpoint pairs, with $0 \leq U(I, q) \leq N$. It is *good* if

$$c(I, q) \leq U(I, q) \leq Ac(I, q)$$

holds at every state. Set $F = \ell$ and $a = \max\{1, A/F\}$.

Fix the complete seed table throughout one refinement. The new random choices are independent of it. Correctness arguments assume that the seed is good. The local procedures are also defined for an inaccurate seed, so the work can be bounded on every seed outcome.

Set

$$P = H^{p_*+20}, \quad T = 2^{\lceil (3/4) \log_2 H \rceil}, \quad S = H/\ell^2, \quad L_0 = 32F, \quad L = 128F.$$

The parameter P controls the endpoint grids. We use T propagation steps followed by S refinement steps. The constants L_0 and L are endpoint-cone slopes. The remaining parameters are

$$\begin{aligned} \delta &= \ell^{-10}, & \kappa &= \tau = \ell^{-4}, & \theta &= \ell^{-40}, \\ \gamma &= \ell^{-20}, & \eta &= \ell^5, & Q_1 &= \ell^{80}. \end{aligned}$$

Here δ controls clipping, κ weights a decrease in child cost, and τ keeps the predicted velocity below one. The parameter θ controls band width and action rounding. Group estimates have additive tolerance γb , the online objective has entropy weight η , and Q_1 controls the number of sampled children.

All grid spacings are powers of two. Integer tests compute the parameters, including the ceilings and floors in their exponents. In this branch, S is a positive integer.

The procedures for one refinement use an immutable record C . It stores the pass and copy identifiers, the seed descriptor $C.U$, the factor $C.A = A$, and $C.a = a$. A later update of the current seed does not change this record. In a cache key, C denotes only its pass and copy identifiers; its fields are stored once.

Lemma 4.2. *The parameters satisfy*

$$H \geq 2^{\ell/2}, \quad \ell^{200} \leq H^{1/100}, \quad p_* + 21 \leq \sqrt{\ell}.$$

Proof. The definition of ℓ gives $\log_2 H \geq \ell/2$. At $\ell \geq 2^{1000}$, we have $200 \log_2 \ell \leq \ell/200$ and $(\log_2 \ell)^{1/4} + 21 \leq \sqrt{\ell}$. The right sides grow faster than the left sides beyond this cutoff. These inequalities prove the other two bounds. $\square$

Algorithm 8 Large-branch parameters and shared data

```
1: function INITIALIZEPARAMETERS( $x, y, N, H, \ell$ )
2:    $p_* \leftarrow \max\{j \in \mathbb{Z}_{\geq 0} : j^4 \leq \log_2 \ell\}$ 
3:    $B \leftarrow H^{p_*}; A_{\text{init}} \leftarrow H^{p_*+4}; Q_0 \leftarrow H^4; \lambda \leftarrow 1/(64H)$ 
4:    $M \leftarrow 2^{\lfloor (\log_2 H)/20 \rfloor}; P \leftarrow H^{p_*+20}; F \leftarrow \ell$ 
5:    $T \leftarrow 2^{\lceil (3/4) \log_2 H \rceil}; S \leftarrow H/\ell^2$ 
6:    $L_0 \leftarrow 32F; L \leftarrow 128F$ 
7:    $\delta \leftarrow \ell^{-10}; \kappa \leftarrow \ell^{-4}; \tau \leftarrow \ell^{-4}; \theta \leftarrow \ell^{-40}$ 
8:    $\gamma \leftarrow \ell^{-20}; \eta \leftarrow \ell^5; Q_1 \leftarrow \ell^{80}$ 
9:    $J \leftarrow \min\{j \geq 0 : M^j \geq \max\{1, |x|\}\}$ 
10:   $I_{\text{root}} \leftarrow ((0, |x|], \text{root}, J)$ 
11:  Sort the pairs  $(y_j, j)$  lexicographically to build the occurrence index
12:  Initialize the empty cache  $\mathcal{D}$ 
13:  Store the parameters, occurrence index, and implicit root for all procedures
14: end function
```

4.2 Cost cones and their local evaluation

The number a is an integer power of two. Define

$$\tilde{U}(I, q) = \max\{\lceil U(I, q)/a \rceil, |z(I, q)|\}.$$

Since $Fc(I, q)$ is an integer, rounding does not spoil the bounds

$$\max\{c(I, q)/a, |z(I, q)|\} \leq \tilde{U}(I, q) \leq Fc(I, q).$$

These bounds do not give endpoint regularity: neighboring seed entries can have quite different overestimates. We replace the scaled table by a lower envelope of cones. The charge $L_0|q - r|_1$ for moving away from a center r preserves the lower bounds c/a and $|z|$, while making the envelope Lipschitz. To evaluate it with few seed queries, we retain centers on grids at several cost scales. A grid at the scale of the true cost will provide a nearby inexpensive center; its value will then force every minimizing center to be nearby as well.

Definition 4.3. The scales b are the powers of two from 1 to the largest power of two at most N . At scale b , put $g_b = \max\{1, b/P\}$. A grid state has both coordinates in $g_b\mathbb{Z} \cap [0, |y|]$, in nondecreasing order. The state $r_b(q)$ rounds both coordinates of q down to this grid. All these spacings are integers. If $g_b = 1$, rounding is exact; otherwise

$$|q - r_b(q)|_1 < 2b/P.$$

Algorithm 9 Endpoint pairs on an implicit grid

```
1: function GRIDPAIRS( $g, [a_-, a_+], [b_-, b_+]$ )
2:    $A \leftarrow \{kg : k \in \mathbb{Z}, 0 \leq kg \leq |y|, a_- \leq kg \leq a_+\}$ 
3:    $B' \leftarrow \{kg : k \in \mathbb{Z}, 0 \leq kg \leq |y|, b_- \leq kg \leq b_+\}$ 
4:   Generate  $A$  and  $B'$  from their smallest and largest integer indices
5:   return  $\{(a, b) : a \in A, b \in B', a \leq b\}$  in the fixed order
6: end function
```

At a nonleaf I , consider all cones

$$q \mapsto \tilde{U}(I, r) + L_0|q - r|_1$$

whose centers are grid states at some scale b with $b/2 \leq U(I, r) \leq 4Ab$. Include also the cone at every state r with $U(I, r) = 0$, whether or not it occurs on one of these grids. Define $w_0(I, q)$ to be their minimum. At a leaf set $w_0(I, q) = c(I, q)$. The exact leaf rule will be used for every later table as well.

Lemma 4.4. *For the fixed seed in Definition 4.1,*

$$\max\{c/a, |z|\} \leq w_0 \leq 2Fc, \quad \text{Lip}(w_0(I, \cdot)) \leq L_0.$$

At a nonleaf, the following local rule computes this global minimum exactly. If $U(I, q) = 0$, return zero. Otherwise omit the zero cones, enumerate scales

$$U(I, q)/(16A^2) \leq b \leq 4AU(I, q),$$

and enumerate grid centers at sup distance at most $U(I, q)$ from q , retaining the eligibility test $b/2 \leq U(I, r) \leq 4Ab$.

Proof. Each cone dominates $c(q)/a$ and $|z(q)|$ by the scaled-seed bounds and the endpoint Lipschitz bounds. Each has Lipschitz constant L_0 , so their minimum does too. Leaves satisfy these claims directly.

If $c(I, q) = 0$, the seed has $U(I, q) = 0$, and its own cone gives zero. For $c = c(I, q) > 0$, choose $b \leq c < 2b$ and set $r = r_b(q)$, $e = |q - r|_1$. By Definition 4.3,

$$c(I, r) \geq b - 2b/P \geq b/2, \quad c(I, r) < 2b + 2b/P \leq 4b.$$

Thus r is an eligible center. Its cone has value at most

$$Fc(I, r) + L_0e \leq Fc + (F + L_0)e \leq Fc + 66Fb/P \leq 2Fc.$$

Any minimizing cone at a positive-cost state consequently has center distance at most $2Fc/L_0 = c/16$. Its center's true cost lies in $[15c/16, 17c/16]$. A zero center cannot minimize, since its distance would be at least c . For a minimizing nonzero center, eligibility gives

$$b \geq \frac{U(I, r)}{4A} \geq \frac{15c}{64A} \geq \frac{U(I, q)}{16A^2}, \quad b \leq 2U(I, r) \leq \frac{17Ac}{8} \leq 4AU(I, q).$$

Its sup distance is at most $c/16 \leq U(I, q)$. Thus the local list contains a global minimizer, and every member of the local list belongs to the global family. The two minima agree. $\square$

We use this local rule as the executable definition even when the seed is inaccurate, returning N if its nonzero branch has no candidate. Its work remains finite without any seed guarantee. Indeed, the scale interval has ratio at most $64A^3$, and $U(I, q)/b \leq 16A^2$ there. The local list contains $O((1 + PA^2)^2(1 + \log A)) = H^{O(p_*+1)}$ raw seed queries. The returned values are nonnegative integers of magnitude $O((L_0 + 1)N)$. Global cone identities and Lipschitz continuity are asserted only for a seed satisfying Definition 4.1.

Algorithm 10 Initial cone evaluation

```
1: function INITIALCONE( $C, I, q$ )
2:   if  $I$  is a leaf then
3:     return LEAFCOST( $I, q$ )
4:   end if
5:    $u_q \leftarrow \text{READ}(C.U, I, q)$ 
6:   if  $u_q = 0$  then
7:     return 0
8:   end if
9:    $v_{\min} \leftarrow +\infty$ 
10:  for dyadic  $b \in [1, N] \cap [u_q/(16C.A^2), 4C.Au_q]$  do
11:     $g \leftarrow \max\{1, b/P\}$ 
12:     $\mathcal{R} \leftarrow \text{GRIDPAIRS}(g, [q^- - u_q, q^- + u_q], [q^+ - u_q, q^+ + u_q])$ 
13:    for  $r \in \mathcal{R}$  do
14:       $u_r \leftarrow \text{READ}(C.U, I, r)$ 
15:      if  $b/2 \leq u_r \leq 4C.Ab$  then
16:         $\tilde{u}_r \leftarrow \max\{\lceil u_r/C.a \rceil, |z(I, r)|\}$ 
17:         $v_{\min} \leftarrow \min\{v_{\min}, \tilde{u}_r + L_0|q - r|_1\}$ 
18:      end if
19:    end for
20:  end for
21:  if  $v_{\min} = +\infty$  then
22:    return  $N$ 
23:  end if
24:  return  $v_{\min}$ 
25: end function
```

4.3 Wide actions and Bellman propagation

The next operator combines child estimates into a parent estimate. We first allow each child to choose its target independently, paying the connection charge for gaps and overlaps. Its minimum over earlier child tables will remain an available parent option throughout the refinement; the more restricted choices will use current child tables. Extending each action cost by an endpoint cone retains spatial regularity. Repeated propagation will also supply the average cost table used to predict child alignments.

Let a nonleaf $I = (l, u]$ have child cuts $l = l_0 \leq l_1 \leq \dots \leq l_M = u$, so $I_i = (l_{i-1}, l_i]$.

Definition 4.5. A *representative* is a pair (b, r) with r a grid state at scale b and

$$b/(4a) \leq w_0(I, r) \leq 8Fb.$$

Write $p_0 = r^- - l$. The wide action set $\mathcal{A}_{b,r}$ consists of lists $\alpha = (s_1, \dots, s_M)$ of valid child grid states satisfying

$$|s_i^- - l_{i-1} - p_0| \leq 10b, \quad |s_i^+ - l_i - p_0| \leq 10b.$$

Consecutive target intervals need not meet. Their discrepancy is paid for by the connection charge $\text{conn}_r(\alpha)$ defined in Section 2. For child tables f , put

$$V_r(\alpha; f) = \text{conn}_r(\alpha) + \sum_{i=1}^M f(I_i, s_i),$$
$$\mathcal{B}_I(f)(q) = \min \left\{ w_0(I, q), \min_{(b,r)} \left[L|q - r|_1 + \min_{\alpha \in \mathcal{A}_{b,r}} V_r(\alpha; f) \right] \right\}.$$

Every minimum over an empty set is $+\infty$.

The operator $\mathcal{B}_I$ is monotone in its child tables and is L -Lipschitz in q . It also preserves the lower bound $\max\{c/a, |z|\}$. For the latter assertion, if each child satisfies the lower bound, then

$$V_r(\alpha; f) \geq \frac{\text{conn}_r(\alpha) + \sum_i c(I_i, s_i)}{a} \geq c(I, r)/a,$$

while the net-change-of-shift inequality gives $V_r(\alpha; f) \geq |z(I, r)|$. Endpoint Lipschitz continuity extends both bounds along the cone, and w_0 satisfies them too.

Algorithm 11 Wide child states and gathered costs

```

1: function WIDESTATES( $I, b, r$ )
2:    $(I_i)_{i=1}^M \leftarrow \text{CHILDREN}(I, M)$ ;  $g \leftarrow \max\{1, b/P\}$ 
3:    $p_0 \leftarrow r^- - l$ 
4:   for  $i = 1, \dots, M$  do
5:      $\mathcal{S}_i \leftarrow \text{GRIDPAIRS}(g, [l_{i-1} + p_0 - 10b, l_{i-1} + p_0 + 10b], [l_i + p_0 - 10b, l_i + p_0 + 10b])$ 
6:   end for
7:   return  $(I_i, \mathcal{S}_i)_{i=1}^M$ 
8: end function
9: function WIDECOSTS( $I, b, r, R$ )
10:   $(I_i, \mathcal{S}_i)_{i=1}^M \leftarrow \text{WIDESTATES}(I, b, r)$ 
11:  for each  $i$  and each distinct  $s \in \mathcal{S}_i$  do
12:     $f_i(s) \leftarrow \text{READ}(R, I_i, s)$ 
13:  end for
14:  return  $(I_i, \mathcal{S}_i, f_i)_{i=1}^M$ 
15: end function
16: function WIDEMINIMUM( $r, \mathcal{F}$ )
17:  Unpack  $\mathcal{F} = (I_i, \mathcal{S}_i, f_i)_{i=1}^M$ 
18:   $Z \leftarrow +\infty$ 
19:  for  $\alpha \in \mathcal{S}_1 \times \dots \times \mathcal{S}_M$  do
20:     $Z \leftarrow \min\{Z, \text{conn}_r(\alpha) + \sum_i f_i(s_i(\alpha))\}$ 
21:  end for
22:  return  $Z$ 
23: end function

```

Definition 4.6. Define the warmup tables at nonleaves by

$$w_j(I, q) = \mathcal{B}_I(w_{j-1})(q) \quad (1 \leq j \leq T), \quad m = \frac{1}{T} \sum_{j=0}^{T-1} w_j, \quad d_0 = w_T.$$

Here the operator receives the indicated tables on children.

Use the descriptors $\mathbf{W}(C, j)$ for w_j and $\mathbf{M}(C)$ for m . An average entry has key $(\mathbf{M}(C), I, q)$. The descriptor $\mathbf{D}(C, 0)$ is an alias for $\mathbf{W}(C, T)$, so the two names request the same stored value.

Lemma 4.7. *For a good seed, the warmup tables are nonincreasing and satisfy*

$$\max\{c/a, |z|\} \leq d_0 \leq m \leq w_0 \leq 2Fc, \quad \mathcal{B}_I(d_0) \leq d_0.$$

The tables w_j and m are L -Lipschitz. For every representative (b, r) and parent state q ,

$$m(I, q) - \frac{w_0(I, q) - w_T(I, q)}{T} \leq L|q - r|_1 + \min_{\alpha \in \mathcal{A}_{b,r}} V_r(\alpha; m).$$

Proof. The Bellman operator is monotone, preserves the lower bounds, and satisfies $w_1 \leq w_0$. Induction gives the bounds on w_j , d_0 , and m . Applying monotonicity once more gives $\mathcal{B}_I(d_0) \leq d_0$. Every propagated table is a minimum of L -Lipschitz cones and w_0 , so it is L -Lipschitz.

To prove the averaging bound, fix a representative and a wide action. Average $w_j(I, q) \leq L|q - r|_1 + V_r(\alpha; w_{j-1})$ over $1 \leq j \leq T$. The left side equals $m - (w_0 - w_T)/T$. The child averages on the right equal m . Now minimize over the fixed action. $\square$

The average m differs from the next T -term average by only $(w_0 - w_T)/T$. This is the averaging error in the parent-child comparison.

Algorithm 12 Warmup tables and their average

```

1: function WARMUP( $C, I, q, j$ )
2:   if  $I$  is a leaf then
3:     return LEAFCOST( $I, q$ )
4:   end if
5:   if  $j = 0$  then
6:     return INITIALCONE( $C, I, q$ )
7:   end if
8:   return BELLMAN( $C, I, q, W(C, j - 1)$ )
9: end function
10: function MEAN( $C, I, q$ )
11:   if  $I$  is a leaf then
12:     return LEAFCOST( $I, q$ )
13:   end if
14:    $v_{\text{sum}} \leftarrow 0$ 
15:   for  $j = 0, \dots, T - 1$  do
16:      $v_{\text{sum}} \leftarrow v_{\text{sum}} + \text{READ}(W(C, j), I, q)$ 
17:   end for
18:   return  $v_{\text{sum}}/T$ 
19: end function

```

Table	Role	Bounds for a good seed
U	Entrywise seed	$c \leq U \leq Ac$
$\tilde{U}$	Scaled seed	$\max\{c/a, z \} \leq \tilde{U} \leq Fc$
w_0	Initial cone envelope	$\max\{c/a, z \} \leq w_0 \leq 2Fc$
w_j	Propagated table	$w_j \leq w_{j-1}$
m	Average of $w_0, \dots, w_{T-1}$	$d_0 \leq m \leq w_0$
d_0	Starting refinement table	$d_0 = w_T$

Table 1: Tables used to regularize and propagate one fixed seed.

4.4 Local evaluation of later cones

A cone that improves on $w_0(I, q)$ must have a nearby center and a comparable scale. The following rule evaluates a global minimum by a local search. We enumerate whole cells, so a choice shared by a cell can be defined independently of the state that first requests it.

Lemma 4.8. *Fix a good seed as in Definition 4.1. Take a minimum of $w_0(I, q)$ and any family of cones of slope L with nonnegative base costs at representatives (b, r) . If $w_0(I, q) = 0$, its value is*

zero. Otherwise it suffices to enumerate scales

$$w_0(I, q)/(16F) \leq b \leq 8aw_0(I, q)$$

and the following whole cells at each scale. Partition endpoint-pair space into half-open cells of side θb on each axis, with lower boundaries at multiples of θb . Enumerate the cells intersecting the closed sup square of radius $w_0(I, q)$ about q , and all valid grid representatives in these cells.

Proof. Write $w = w_0(I, q) > 0$. A competitive cone has value at most w and $|q - r|_1 \leq w/L$. Since $L_0/L = 1/4$, $3w/4 \leq w_0(I, r) \leq 5w/4$. Representative eligibility yields

$$b \geq 3w/(32F) > w/(16F), \quad b \leq 5aw < 8aw.$$

The representative belongs to the enumerated square and hence to an enumerated cell. Conversely, enumerating a whole cell only adds cones already in the global family. This proves exactness. $\square$

For arbitrary local w_0 , the scale count in Lemma 4.8 is $O(1 + \log(aF))$, polynomial in ℓ . There are $O((1 + F/\theta)^2)$ cells per scale and $O((1 + P)^2)$ grid states per cell. Indeed $w_0(I, q)/b \leq 16F$, and $\theta b/g_b \leq P$. Cells and grid points are generated by ranges of integer indices, without scanning unrelated positions. These count bounds do not assume a correct seed. Empty lists give the w_0 option.

Algorithm 13 Local scales, whole cells, and eligible representatives

```

1: function LOCALSCALES( $C, I, q$ )
2:    $w \leftarrow \text{READ}(\mathbf{W}(C, 0), I, q)$ 
3:   return the dyadic  $b \in [1, N] \cap [w/(16F), 8C.aw]$ 
4: end function
5: function LOCALCELLS( $b, q, w$ )
6:    $d_{\text{cell}} \leftarrow \theta b$ 
7:   for each coordinate  $\xi \in \{-, +\}$  do
8:      $k_-^\xi \leftarrow \lfloor \max\{0, q^\xi - w\}/d_{\text{cell}} \rfloor$ 
9:      $k_+^\xi \leftarrow \lfloor \min\{|y|, q^\xi + w\}/d_{\text{cell}} \rfloor$ 
10:  end for
11:  return all index pairs in  $[k_-^-, k_+^-] \times [k_-^+, k_+^+]$ 
12: end function
13: function ISREPRESENTATIVE( $C, I, b, r$ )
14:    $w_r \leftarrow \text{READ}(\mathbf{W}(C, 0), I, r)$ 
15:   return the test  $b/(4C.a) \leq w_r \leq 8Fb$ 
16: end function
17: function CELLREPRESENTATIVES( $C, I, b, (k^-, k^+)$ )
18:    $g \leftarrow \max\{1, b/P\}$ ;  $d_{\text{cell}} \leftarrow \theta b$ 
19:    $\mathcal{R} \leftarrow \text{GRIDPAIRS}(g, [k^-d_{\text{cell}}, (k^- + 1)d_{\text{cell}}], [k^+d_{\text{cell}}, (k^+ + 1)d_{\text{cell}}])$ 
20:   Discard  $r$  on either upper cell boundary, so the cell is half-open
21:   return the  $r \in \mathcal{R}$  for which  $\text{ISREPRESENTATIVE}(C, I, b, r)$  is true
22: end function

```

Algorithm 14 Local Bellman evaluation

```
1: function BELLMAN( $C, I, q, R$ )
2:    $w \leftarrow \text{READ}(\text{W}(C, 0), I, q)$ ;  $v_{\min} \leftarrow w$ 
3:   if  $w = 0$  then
4:     return 0
5:   end if
6:   for  $b \in \text{LOCALSCALES}(C, I, q)$  do
7:     for  $c_{\text{cell}} \in \text{LOCALCELLS}(b, q, w)$  do
8:       for  $r \in \text{CELLREPRESENTATIVES}(C, I, b, c_{\text{cell}})$  do
9:          $\mathcal{F} \leftarrow \text{WIDECOSTS}(I, b, r, R)$ 
10:         $Z \leftarrow \text{WIDEMINIMUM}(r, \mathcal{F})$ 
11:         $v_{\min} \leftarrow \min\{v_{\min}, L|q - r|_1 + Z\}$ 
12:      end for
13:    end for
14:  end for
15:  return  $v_{\min}$ 
16: end function
```

We have now obtained nonincreasing propagated estimates and the cost proxy m . The cost proxy m can now predict where a shared child alignment crosses each source cut. Wide actions use earlier tables, while narrow sets of shared alignments restrict the calls to current tables.

5 Predicted bands and the refined tables

Starting from the fixed seed and the table $d_0 = w_T$, we now define $d_1, \dots, d_S$. The construction proceeds by node height: when defining the value at a parent and index t , all child tables through index t are already defined. This is a mathematical ordering, not a requirement to evaluate those tables in full. The algorithm will request only the entries needed by the choices below.

There are two ways to choose a current-index action. A shared sample compares rounded actions at nearby representatives; extending the chosen cost by a cone gives a spatially regular estimate. An online rule uses earlier child values to choose an action with small expected accumulated cost relative to every fixed action in a predicted band. The parent update combines these two estimates. We first construct the band and its child cost bounds, then specify the online rule and the shared samples.

Write $\text{D}(C, t)$ for the descriptor of d_t in copy C . Band data, group choices, and online actions are also stored under the keys shown in their procedures. A skipped band and an empty group are stored results. A group key omits the requesting state, so all requests for the same cell reuse its choice.

5.1 A band predicted from prefix costs

Definition 5.1. At a representative (b, r) of a nonleaf, define

$$Z = \min_{\alpha \in \mathcal{A}_{b,r}} V_r(\alpha; m).$$

Skip the band computations at this representative unless Z is finite and strictly positive. Otherwise set $\hat{v} = z(I, r)/Z$, the net shift per unit of predicted cost. If shift accumulated proportionally to this cost along an alignment, a prefix of cost C would end at shift $p_0 + \hat{v}C$. We use this relation to predict the shift at each child cut. We call $|\hat{v}| \leq 1 - \tau$ the *velocity test*, and skip unless it holds.

For an internal cut $1 \leq j < M$ and a real shift p , let $\Phi_j(p)$ be the minimum prefix cost using the first j child restrictions of the wide action set. More explicitly, minimize

$$|s_1^- - r^-| + \sum_{i=1}^j m(I_i, s_i) + \sum_{i < j} |s_{i+1}^- - s_i^+| + |s_j^+ - (l_j + p)|$$

over such prefix states. The wide action family is nonempty whenever the band reaches this step, so these prefix minima are finite. Each function in the minimum is 1-Lipschitz in p , hence so is Φ_j .

The prefix cost depends on its proposed ending shift. The predicted shift satisfies the approximate fixed-point relation $p = p_0 + \hat{v}\Phi_j(p)$. The velocity test keeps the Lipschitz constant of $\hat{v}\Phi_j$ below one, so all sufficiently accurate solutions lie in a short interval. The band below imposes this relation at every cut.

Let $\mathcal{C}_{b,r}$ be the following set of shared-endpoint actions on the scale- b grid. The outside endpoints are exactly r , and consecutive child endpoints agree. Every endpoint shift is within $6b$ of $p_0 = r^- - l$. At each internal cut, its shift p satisfies

$$|p - p_0 - \hat{v}\Phi_j(p)| \leq \tau\theta b/M.$$

All child target states must be valid, so these shared endpoints are in nondecreasing order. Skip if $\mathcal{C}_{b,r}$ is empty.

The possible shifts at any fixed cut have diameter at most $2\theta b/M$. In fact two allowed shifts obey

$$|p - p'| \leq 2\tau\theta b/M + |\hat{v}| |\Phi_j(p) - \Phi_j(p')| \leq 2\tau\theta b/M + (1 - \tau)|p - p'|.$$

No convexity of the allowed shift set is needed.

We need a uniform cost bound at each child for both action selection and rounding. For a nonempty band, let h_i be the smallest power of two at least

$$\max \left\{ b/M, \max_{\alpha \in \mathcal{C}_{b,r}} w_0(I_i, s_i(\alpha)) \right\}, \quad h_\Sigma = \sum_{i=1}^M h_i.$$

Powers of two below one are allowed here. Every band state at child i has w_0 value at most h_i . These bounds will normalize the online costs and determine a common rounding spacing for each child scale. Skip if $h_\Sigma > 64Fb$. Choose a canonical band action in a fixed ordering and round its two coordinates separately at child i down to multiples of $\max\{1, \theta h_i\}$. Denote the resulting action by $\beta(b, r)$. Different children may round a shared endpoint to different positions, so β can have gaps or overlaps. It is used as a wide action with its connection charge, rather than as a band action. Finally, skip unless it belongs to $\mathcal{A}_{b,r}$ and

$$|z(I_i, s_i(\beta))| \leq 2h_i \quad (1 \leq i \leq M).$$

Representatives passing all these tests will be called unskipped. These tests affect only the band and rounded-action computations; they never remove an action or representative from $\mathcal{B}_I$.

Under the fixed good seed, the last two guards on β pass whenever the h_Σ test passes. Indeed, $\theta h_i \geq \theta b/M \geq b/P$. All relevant quantities are powers of two, so the rounded spacing is a multiple of g_b . Rounding preserves the order of each child's two endpoints and the range $[0, |y|]$. If its spacing is one, its error is zero; otherwise each coordinate error is less than θh_i . Since $|z| \leq w_0 \leq h_i$ before rounding,

$$|z(I_i, s_i(\beta))| \leq h_i + 2\theta h_i \leq 2h_i.$$

The endpoint shift radius after rounding is at most $6b + 64F\theta b < 10b$.

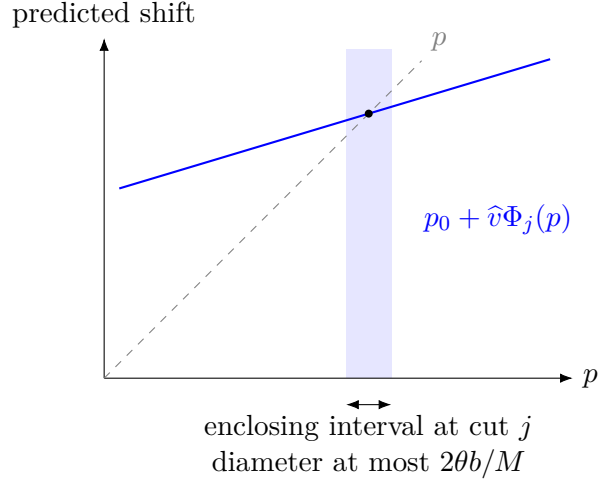


Figure 2: The band test at one cut, shown schematically. The predicted-shift function is $(1 - \tau)$ -Lipschitz. Shifts satisfying $|p - p_0 - \hat{v}\Phi_j(p)| \leq \tau\theta b/M$ lie in an interval of diameter at most $2\theta b/M$. The algorithm retains only valid grid shifts and shared-endpoint actions.

Algorithm 15 A prefix minimum at a candidate target boundary

```

1: function PREFIXCOST( $r, j, e, \mathcal{F}$ )
2:   Unpack  $\mathcal{F} = (I_i, \mathcal{S}_i, f_i)_{i=1}^M$ 
3:    $\phi \leftarrow +\infty$ 
4:   for  $(s_1, \dots, s_j) \in \mathcal{S}_1 \times \dots \times \mathcal{S}_j$  do
5:      $v \leftarrow |s_1^- - r^-| + \sum_{i=1}^j f_i(s_i) + \sum_{i < j} |s_{i+1}^- - s_i^+| + |s_j^+ - e|$ 
6:      $\phi \leftarrow \min\{\phi, v\}$ 
7:   end for
8:   return  $\phi$ 
9: end function

```

Algorithm 16 Constructing and rounding a predicted band

```
1: function BANDDATA( $C, I, b, r$ )
2:    $\mathcal{F} = (I_i, \mathcal{S}_i, f_i)_{i=1}^M \leftarrow \text{WIDECOSTS}(I, b, r, M(C))$ 
3:    $Z \leftarrow \text{WIDEMINIMUM}(r, \mathcal{F})$ 
4:   if  $Z \notin (0, +\infty)$  then
5:     return SKIPPED
6:   end if
7:    $\hat{v} \leftarrow z(I, r)/Z$ ;  $p_0 \leftarrow r^- - l$ ;  $g \leftarrow \max\{1, b/P\}$ 
8:   if  $|\hat{v}| > 1 - \tau$  or  $|r^+ - u - p_0| > 6b$  then
9:     return SKIPPED
10:  end if
11:   $E_0 \leftarrow \{r^-\}$ ;  $E_M \leftarrow \{r^+\}$ 
12:  for  $j = 1, \dots, M-1$  do
13:     $E_j \leftarrow \emptyset$ 
14:    for grid boundaries  $e \in [0, |y|] \cap g\mathbb{Z}$  with  $|e - l_j - p_0| \leq 6b$  do
15:       $\phi \leftarrow \text{PREFIXCOST}(r, j, e, \mathcal{F})$ 
16:      if  $|e - l_j - p_0 - \hat{v}\phi| \leq \tau\theta b/M$  then
17:        Insert  $e$  into  $E_j$ 
18:      end if
19:    end for
20:  end for
21:   $\mathcal{C} \leftarrow \{((e_{i-1}, e_i))_{i=1}^M : e_j \in E_j, e_0 \leq \dots \leq e_M\}$ 
22:  if  $\mathcal{C} = \emptyset$  then
23:    return SKIPPED
24:  end if
25:  for  $i = 1, \dots, M$  do
26:    Gather  $w_i(s) \leftarrow \text{READ}(W(C, 0), I_i, s)$  once for each child state in  $\mathcal{C}$ 
27:     $h_i \leftarrow$  the least dyadic number at least  $\max\{b/M, \max_s w_i(s)\}$ 
28:  end for
29:  if  $\sum_i h_i > 64Fb$  then
30:    return SKIPPED
31:  end if
32:   $\alpha \leftarrow$  the first action in  $\mathcal{C}$ 
33:  for  $i = 1, \dots, M$  do
34:     $D_i \leftarrow \max\{1, \theta h_i\}$ 
35:     $\beta_i \leftarrow (D_i \lfloor s_i^-(\alpha)/D_i \rfloor, D_i \lfloor s_i^+(\alpha)/D_i \rfloor)$ 
36:  end for
37:  if some  $\beta_i \notin \mathcal{S}_i$  or  $|z(I_i, \beta_i)| > 2h_i$  then
38:    return SKIPPED
39:  end if
40:  return  $(\mathcal{C}, (h_i)_{i=1}^M, \beta)$ 
41: end function
```

5.2 An entropy inequality with fixed block masses

Exponential weighting and its potential analysis have a classical role in online learning; see Freund and Schapire [13]. Our predicted objective has the form of optimistic regularized leader, as studied by Rakhlin and Sridharan [26]. For our action vectors, each child receives a fixed total probability mass. Cost decrements may differ greatly between children, but their variation within a child is small. The following deterministic lemma isolates the reason that this weaker variation bound suffices.

Lemma 5.2. *Let $\mathcal{Q}$ be a nonempty compact convex set of probability vectors in $\mathbb{R}^d$, containing a vector with every coordinate positive. Partition the coordinates into blocks, and suppose that the total mass in each block is the same for every vector in $\mathcal{Q}$. For an integer $S \geq 1$, let $f_0, \dots, f_S \in [0, 1]^d$ be coordinatewise nonincreasing. For $\eta > 0$ and $0 < \kappa < 1/2$, set*

$$\hat{g}_t = -f_{t-1}, \quad \Delta_t = f_{t-1} - f_t, \quad u_t = \hat{g}_t + (1 - 2\kappa)\Delta_t.$$

Suppose that the range of Δ_t within each block is at most κ/η for every $1 \leq t \leq S$. Let ρ_t maximize

$$\mathcal{H}(\rho) + \eta \left\langle \sum_{j < t} u_j + \hat{g}_t, \rho \right\rangle \quad \text{over } \rho \in \mathcal{Q}, \quad \mathcal{H}(\rho) = -\sum_k \rho_k \ln \rho_k,$$

with $0 \ln 0 = 0$. For every $\rho^ \in \mathcal{Q}$ and every integer $0 \leq s \leq S$,*

$$\sum_{t=1}^s \langle f_t + \kappa \Delta_t, \rho_t \rangle \leq \sum_{t=1}^s \langle f_t, \rho^* \rangle + \frac{\ln d}{\eta} + 2\kappa.$$

For any $\zeta \geq 0$, replacing each ρ_t on the left by a vector within ζ in ℓ_1 distance adds at most $s\zeta$ to the bound.

Proof. The monotone cost vectors satisfy, coordinatewise,

$$\Delta_t \geq 0, \quad \sum_{t=1}^S \Delta_t = f_0 - f_S \leq 1.$$

Define the potential

$$\Psi(v) = \max_{\rho \in \mathcal{Q}} \{ \mathcal{H}(\rho) + \eta \langle v, \rho \rangle \}.$$

Compactness gives a maximizer, and strict concavity of entropy makes it unique. It is also positive. To see this, move a small distance s from a vector with a zero coordinate toward the positive feasible vector. Entropy increases by a positive multiple of $s \ln(1/s) + O(s)$, whereas the linear term changes by only $O(s)$. Thus a vector with a zero coordinate cannot maximize the objective. At a positive maximizer p , first-order optimality along feasible segments gives

$$\langle \nabla \mathcal{H}(p) + \eta v, \rho - p \rangle \leq 0 \quad (\rho \in \mathcal{Q}).$$

Since $\mathcal{H}(\rho) - \mathcal{H}(p) = \langle \nabla \mathcal{H}(p), \rho - p \rangle - D_{\text{KL}}(\rho \| p)$, we obtain the precise penalty

$$\mathcal{H}(\rho) + \eta \langle v, \rho \rangle \leq \Psi(v) - D_{\text{KL}}(\rho \| p).$$

Here $D_{\text{KL}}(\rho \| p) = \sum_k \rho_k \ln(\rho_k/p_k)$, with zero terms interpreted as zero. The penalty holds on $\mathcal{Q}$ itself, even when its feasible directions are restricted.

Put $v_{t-1} = \sum_{j < t} u_j$. Evaluating the old objective at the maximizer ρ_t of the new one gives

$$\Psi(v_{t-1} + \hat{g}_t) - \Psi(v_{t-1}) \leq \eta \langle \hat{g}_t, \rho_t \rangle.$$

This inequality remains valid when the increment is negative. To add the remaining vector $(1-2\kappa)\Delta_t$, let μ be the vector whose coordinates are the minimum of Δ_t in their block, and write $v' = \Delta_t - \mu$. Then $0 \leq \eta v'_k \leq \kappa$ and $\mu \geq 0$. Moreover $\langle \mu, \rho \rangle$ is the same for every $\rho \in \mathcal{Q}$, because the block masses are fixed. Thus the possibly large block minima contribute exactly the same linear increment at every feasible vector; only the small within-block remainder needs an exponential estimate. The relative-entropy penalty bounds the remaining potential increment by

$$\eta(1-2\kappa) \langle \mu, \rho_t \rangle + \ln \sum_k \rho_{t,k} \exp(\eta(1-2\kappa)v'_k).$$

To justify the logarithmic term, extend the supremum with its relative entropy penalty to the full probability simplex and normalize the weights $\rho_{t,k} \exp(\eta(1-2\kappa)v'_k)$. The resulting objective equals the displayed logarithm minus relative entropy to these normalized weights, whose nonnegativity proves the bound.

For $0 \leq x \leq \kappa \leq 1/2$,

$$e^{(1-2\kappa)x} \leq 1 + (1-\kappa)x.$$

Indeed the derivative of $\ln(1 + (1-\kappa)x) - (1-2\kappa)x$ is

$$\frac{\kappa - (1-2\kappa)(1-\kappa)x}{1 + (1-\kappa)x} \geq 0,$$

and the expression vanishes at zero. Apply the exponential bound with $x = \eta v'_k \leq \kappa$, and then use $\ln(1+y) \leq y$. Because $\mu \geq 0$, the remaining potential increment is at most $\eta(1-\kappa) \langle \Delta_t, \rho_t \rangle$. Combining the two increments yields

$$\Psi(v_t) - \Psi(v_{t-1}) \leq \eta \langle \hat{g}_t + (1-\kappa)\Delta_t, \rho_t \rangle.$$

Sum through s , use $\Psi(0) \leq \ln d$, and evaluate the last potential at ρ^* . Its entropy is nonnegative. Replacing the coefficient $1-2\kappa$ in the accumulated comparator objective by 1 loses at most

$$2\kappa \left\langle \sum_{t=1}^s \Delta_t, \rho^* \right\rangle \leq 2\kappa$$

by the cumulative decrement bound. Finally,

$$\hat{g}_t + \Delta_t = -f_t, \quad \hat{g}_t + (1-\kappa)\Delta_t = -(f_t + \kappa\Delta_t).$$

Reversing the signs gives the claimed regret bound. The cost vector $f_t + \kappa\Delta_t = \kappa f_{t-1} + (1-\kappa)f_t$ lies in $[0, 1]^d$, so an ℓ_1 error of at most ζ costs at most ζ per term. $\square$

5.3 Finite entropy optimization

We implement the entropy objective by the conditional-gradient method of Frank and Wolfe [12], with the fixed-step form analyzed by Jaggi [15]. Smoothing makes every derivative finite. Rational approximations select vertices, and an exact rational mixture determines the sampled action.

Lemma 5.3. *Let $\mathcal{Q}$ be the convex hull of a nonempty finite list of rational probability vectors in $\mathbb{R}^d$, and suppose that every coordinate is positive in at least one listed vector. Given a rational vector v and an integer $H_{\text{opt}} \geq 4$, put $G_0 = H_{\text{opt}} + d + 2$. A rational algorithm constructs a mixture of the listed vectors whose mean ρ satisfies*

$$\|\rho - \rho_t\|_1 \leq H_{\text{opt}}^{-3},$$

where ρ_t is the unique maximizer of $\mathcal{H}(\rho) + \langle v, \rho \rangle$ on $\mathcal{Q}$. The number of bit operations and the stored mixture size are polynomial in G_0 , the list size, and the maximum numerator or denominator bit length of an input rational. The mixture can be sampled exactly using fair random bits, with expected cost polynomial in the same quantities and with a fixed space bound.

Proof. There is a feasible vector positive in every coordinate, obtained by averaging the list. The maximizer is positive as well: at a zero coordinate, moving a small distance towards that average produces a positive entropy contribution of order $s \ln(1/s)$, which dominates the terms of order s from the other coordinates and the linear objective. Strict concavity gives uniqueness.

Set $\sigma = G_0^{-50}$ and smooth the objective by replacing $-x \ln x$ with $-(x + \sigma) \ln(x + \sigma)$ in each coordinate. Denote the result by $\mathcal{F}_\sigma$. Integrating the derivative of $x \ln x$, including at zero, gives

$$\sup_{\rho \in \mathcal{Q}} |\mathcal{F}_\sigma(\rho) - \mathcal{F}(\rho)| \leq 4d\sqrt{\sigma}.$$

The gradient of $\mathcal{F}_\sigma$ has Euclidean Lipschitz constant at most $1/\sigma$. The diameter of $\mathcal{Q}$ is at most 2.

Start at a listed vertex. At step $j \geq 0$, approximate every coordinate of the gradient to absolute error at most $e_0 = G_0^{-20}$. Enumerate the list and choose a vertex v_j maximizing its scalar product with this approximate gradient, resolving ties by a fixed order. Update

$$\rho^{(j+1)} = (1 - \xi_j)\rho^{(j)} + \xi_j v_j, \quad \xi_j = \frac{2}{j+2}.$$

Here v_j denotes a selected vertex, whereas v without a subscript is the fixed linear coefficient vector. The difference between two probability vectors has ℓ_1 norm at most 2. Consequently the chosen vertex loses at most $2e_0$ in the true linearization, even relative to a feasible mixture. If D_j is the gap from the maximum of $\mathcal{F}_\sigma$, concavity and the gradient bound give

$$D_{j+1} \leq (1 - \xi_j)D_j + 2e_0\xi_j + \frac{2\xi_j^2}{\sigma}.$$

Since $\xi_0 = 1$, this estimate first gives $D_1 \leq 2e_0 + 2/\sigma$, independently of the size of the linear coefficient vector. Induction then gives

$$D_j \leq 2e_0 + \frac{8}{\sigma(j+1)} \quad (j \geq 1).$$

Run $K_{\text{opt}} = G_0^{80}$ steps. By the smoothing bound, the resulting true objective gap is at most

$$2G_0^{-20} + 8G_0^{-30} + 8dG_0^{-25} \leq 12G_0^{-20}.$$

The Hessian of negative entropy is $\text{diag}(1/\rho_k)$, which dominates the identity on the probability simplex. The first-order inequality at the positive maximizer combines with this Hessian bound. The objective gap at any feasible ρ is at least $\frac{1}{2}\|\rho - \rho_t\|_2^2$. The statement at coordinates equal to zero follows by continuity. Hence

$$\|\rho - \rho_t\|_1 \leq \sqrt{d}\|\rho - \rho_t\|_2 \leq \sqrt{24d}G_0^{-10} \leq H_{\text{opt}}^{-3}.$$

We next give an exact representation of every iterate. After $K \geq 1$ steps, the weight of the vertex selected at step j , where $0 \leq j < K$, is

$$\frac{2}{j+2} \prod_{k=j+1}^{K-1} \frac{k}{k+2} = \frac{2(j+1)}{K(K+1)}.$$

The initial vertex has zero remaining weight because $\xi_0 = 1$. Store the sequence of selected vertices, allowing repetitions. The displayed weights sum to one and give the iterate exactly. A common denominator for the input vertices, multiplied by $K(K+1)$, is also a common denominator for its coordinates. One may obtain the input denominator by multiplying all input denominators, whose total bit length is at most the input size. The iterates can be maintained by rational arithmetic of polynomial bit cost, without accumulating the denominators of successive numerical approximations.

It remains to calculate the approximate gradients using finite rational operations. For $r = x + \sigma \in [\sigma, 1 + \sigma]$, put $b_0 = (r - 1)/(r + 1)$. Then $|b_0| \leq 1 - \sigma$, and

$$\ln r = 2 \sum_{k \geq 0} \frac{b_0^{2k+1}}{2k+1}.$$

Truncation after $K_{\log} = G_0^{60}$ terms has absolute error at most

$$\frac{2|b_0|^{2K_{\log}+1}}{(2K_{\log}+1)(1-b_0^2)} \leq \frac{2e^{-2K_{\log}\sigma}}{(2K_{\log}+1)\sigma} < G_0^{-20}.$$

All terms are rational. The linear part of the gradient and its constant entropy derivative term are computed exactly. Thus these finite sums provide the required gradient precision; choosing a vertex requires only rational comparisons.

Computing each rational logarithm sum by direct integer arithmetic also has polynomial bit cost: the lengths of the powers of a rational and of the products of the odd integer denominators are polynomial in $K_{\log}$ and the input bit length. These logarithmic approximations only select vertices; they are not returned as recursive table values.

The same representation gives an exact sampler. Put $K = K_{\text{opt}}$ and sample a uniform integer $R \in \{1, \dots, K(K+1)/2\}$ by drawing uniformly from a range whose size is the least power of two at least $K(K+1)/2$, and rejecting out-of-range values. Each trial succeeds with probability greater than $1/2$. Choose the smallest j with $(j+1)(j+2)/2 \geq R$, by integer binary search, and return v_j . The mixture-weight formula proves exactness, including when the same vertex was selected more than once. The integer has $O(\log K)$ bits. Rejection reuses a fixed buffer, and its expected bit cost is polynomial. This completes the construction. $\square$

Algorithm 17 Rational logarithms for the entropy gradient

```

1: function RATIONALLOG( $r, G_0$ )
2:    $b_0 \leftarrow (r-1)/(r+1)$ ;  $p \leftarrow b_0$ ;  $s \leftarrow 0$ 
3:   for  $k = 0, \dots, G_0^{60} - 1$  do
4:      $s \leftarrow s + p/(2k+1)$ ;  $p \leftarrow p b_0^2$ 
5:   end for
6:   return  $2s$ 
7: end function

```

Algorithm 18 A finite entropy mixture and its exact sampler

Input: An ordered list $\mathcal{V}$ of probability vertices, each tagged with its action

```

1: function ENTROPYMIXTURE( $\mathcal{V}, v, H_{\text{opt}}$ )
2:    $d \leftarrow$  the number of coordinates;  $G_0 \leftarrow H_{\text{opt}} + d + 2$ 
3:    $\sigma \leftarrow G_0^{-50}$ ;  $K \leftarrow G_0^{80}$ ;  $\rho \leftarrow$  the first listed vertex
4:   for  $j = 0, \dots, K-1$  do
5:     for each coordinate  $k$  do
6:        $g_k \leftarrow -\text{RATIONALLOG}(\rho_k + \sigma, G_0) - 1 + v_k$ 
7:     end for
8:      $(\alpha_j, v_j) \leftarrow$  the first tagged vertex maximizing  $\langle g, v_j \rangle$ 
9:      $\rho \leftarrow (1 - 2/(j+2))\rho + (2/(j+2))v_j$ 
10:  end for
11:  return  $(\alpha_j)_{j=0}^{K-1}$ , whose weights are  $2(j+1)/(K(K+1))$ 
12: end function
13: function SAMPLEMIXTURE( $((\alpha_j)_{j=0}^{K-1}, \omega)$ )
14:    $R \leftarrow \text{UNIFORMINT}(K(K+1)/2, \omega)$ 
15:   Find the smallest  $j$  with  $(j+1)(j+2)/2 \geq R$  by integer binary search
16:   return  $\alpha_j$ 
17: end function

```

5.4 Online choices from child histories

The online rule uses child values at indices less than t to select one band action at time t . The fixed-block entropy inequality bounds its accumulated current-index cost against every fixed band action. We encode a mixture of actions by its child-state marginals, with weights chosen to recover the unnormalized sum of child costs.

Definition 5.4. Fix one unskipped band $\mathcal{C} = \mathcal{C}_{b,r}$. Its coordinates are pairs (i, s) , one for each child and each state appearing at that child in a band action. Let $d_{\mathcal{C}}$ be their number. Map each action α to the probability vector $\rho(\alpha)$ that places mass h_i/h_{Σ} at $(i, s_i(\alpha))$ for every child, and zero at its other coordinates. Let $\mathcal{Q} = \text{conv}\{\rho(\alpha) : \alpha \in \mathcal{C}\}$. Every vector in this convex hull has the same total mass h_i/h_{Σ} on the coordinates of child i .

Define coordinate vectors

$$\begin{aligned}\hat{g}_j(i, s) &= -d_{j-1}(I_i, s)/h_i, \\ \Delta_j(i, s) &= (d_{j-1}(I_i, s) - d_j(I_i, s))/h_i, \\ u_j &= \hat{g}_j + (1 - 2\kappa)\Delta_j.\end{aligned}$$

Here $\hat{g}_j$ predicts the negative normalized current cost from the preceding index, and Δ_j is the observed correction:

$$\hat{g}_j(i, s) + \Delta_j(i, s) = -d_j(I_i, s)/h_i, \quad -h_{\Sigma}\langle \hat{g}_j, \rho(\alpha) \rangle = \sum_i d_{j-1}(I_i, s_i(\alpha)).$$

The fixed mass h_i/h_{Σ} in each child block cancels the normalization by h_i . The objective below uses Δ_j only for $j < t$ and uses $\hat{g}_t$, so its required child indices are all strictly earlier than t .

With entropy $\mathcal{H}(\rho) = -\sum_k \rho_k \ln \rho_k$, using $0 \ln 0 = 0$, let ρ_t maximize

$$\mathcal{H}(\rho) + \eta \left\langle \sum_{j < t} u_j + \hat{g}_t, \rho \right\rangle \quad \text{over } \rho \in \mathcal{Q}.$$

The algorithm computes a finite mixture of band actions whose vector of expected weights is within H^{-3} in ℓ_1 distance of ρ_t , then samples that mixture exactly. Use Lemma 5.3 with $H_{\text{opt}} = H$ to construct the mixture by rational arithmetic.

Only the child histories are needed for this rule, not a transcript of earlier queries at the parent. Parent draws may be keyed by (I, q, b, t) in addition to pass and copy identifiers. Group choices and table values are cached on their complete keys, and every first parent draw uses a fresh source independent of the complete child tables.

Algorithm 19 An online choice from earlier child values

```
1: function ONLINECHOICE( $C, I, q, b, t, \omega$ )
2:    $(I_i)_{i=1}^M \leftarrow \text{CHILDREN}(I, M)$ 
3:    $g \leftarrow \max\{1, b/P\}; r \leftarrow (g\lfloor q^-/g \rfloor, g\lfloor q^+/g \rfloor)$ 
4:    $(\mathcal{C}, (h_i), \beta) \leftarrow \text{GET}((\text{Band}, C, I, b, r))$ 
5:    $\mathcal{K} \leftarrow \{(i, s) : s = s_i(\alpha) \text{ for some } \alpha \in \mathcal{C}\}$ 
6:   for each  $(i, s) \in \mathcal{K}$  and  $j = 0, \dots, t-1$  do
7:      $f_j(i, s) \leftarrow \text{READ}(\text{D}(C, j), I_i, s)$ 
8:   end for
9:   for each  $(i, s) \in \mathcal{K}$  do
10:     $v(i, s) \leftarrow -\eta f_{t-1}(i, s)/h_i$ 
11:    for  $j = 1, \dots, t-1$  do
12:       $u_j \leftarrow [-f_{j-1}(i, s) + (1 - 2\kappa)(f_{j-1}(i, s) - f_j(i, s))]/h_i$ 
13:       $v(i, s) \leftarrow v(i, s) + \eta u_j$ 
14:    end for
15:  end for
16:   $h_\Sigma \leftarrow \sum_i h_i$ 
17:  for  $\alpha \in \mathcal{C}$  do
18:     $\rho_\alpha(i, s) \leftarrow (h_i/h_\Sigma)\mathbf{1}_{s=s_i(\alpha)} \text{ on } \mathcal{K}$ 
19:  end for
20:   $\mathcal{V} \leftarrow (\alpha, \rho_\alpha)_{\alpha \in \mathcal{C}}$  in the fixed order
21:   $\mu \leftarrow \text{ENTROPYMIXTURE}(\mathcal{V}, v, H)$ 
22:  return  $\text{SAMPLEMIXTURE}(\mu, \omega)$ 
23: end function
```

5.5 Shared group samples and the update

For each node I , scale b , cell from Lemma 4.8, and time $1 \leq t \leq S$, group all unskipped representatives in that whole cell. The following procedure chooses one member of each nonempty group. It is defined from complete child tables; later we evaluate its required entries on demand.

Sampling and rounding work together here. Sharing the child draws across a cell lets all its representatives reuse the same queried child labels. Rounding then limits how many different states at one such child can occur across the group. This combination, rather than band width alone, bounds the number of current-index recursive calls.

For each power of two h in $[b/M, 64Fb]$, set

$$k_h = \min\{M, \lceil Q_1 M h / b \rceil\}.$$

For each member (b, r) , we estimate the child sum in $V_r(\beta(b, r); d_t)$ by separating the children according to their h_i . If $k_h = M$, include the contribution with $h_i = h$ exactly. Otherwise draw k_h independent uniform child indices with replacement, using the same draws for every member of this group at this scale. For each sampled child include its d_t value if its $h_i = h$, and zero otherwise; multiply the sampled sum by M/k_h . Add the connection charge exactly. Select a member minimizing this estimate, breaking ties by a fixed ordering. Then evaluate its actual action cost using all children. Samples used for different parent groups and times are independent and are independent of descendant randomness.

Rounding and shared child draws work together: a sampled child label can be reused across representatives, and coarse rounding at larger h reduces the number of distinct states at that label.

Algorithm 20 One shared group selection

```
1: function GROUPCHOICE( $C, I, b, c_{\text{cell}}, t, \omega$ )
2:    $(I_i)_{i=1}^M \leftarrow \text{CHILDREN}(I, M)$ 
3:    $\mathcal{G} \leftarrow \emptyset$ 
4:   for  $r \in \text{CELLREPRESENTATIVES}(C, I, b, c_{\text{cell}})$  do
5:      $B_r \leftarrow \text{GET}((\text{Band}, C, I, b, r))$ 
6:     if  $B_r \neq \text{SKIPPED}$  then
7:       Insert  $r$  into  $\mathcal{G}$ 
8:     end if
9:   end for
10:  if  $\mathcal{G} = \emptyset$  then
11:    return EMPTY
12:  end if
13:  for  $r \in \mathcal{G}$  do
14:    Write  $B_r = (C_r, (h_i(r)), \beta(r))$ 
15:     $\hat{V}(r) \leftarrow \text{conn}_r(\beta(r))$ 
16:  end for
17:  for dyadic  $h \in [b/M, 64Fb]$  do
18:     $k_h \leftarrow \min\{M, \lceil Q_1 M h / b \rceil\}$ 
19:    if  $k_h = M$  then
20:      for  $r \in \mathcal{G}$  and  $i$  with  $h_i(r) = h$  do
21:         $\hat{V}(r) \leftarrow \hat{V}(r) + \text{READ}(\text{D}(C, t), I_i, \beta_i(r))$ 
22:      end for
23:    else
24:       $j_s \leftarrow \text{UNIFORMINT}(M, \omega)$  for  $s = 1, \dots, k_h$ 
25:      for  $r \in \mathcal{G}$  and  $s$  with  $h_{j_s}(r) = h$  do
26:         $\hat{V}(r) \leftarrow \hat{V}(r) + (M/k_h) \text{READ}(\text{D}(C, t), I_{j_s}, \beta_{j_s}(r))$ 
27:      end for
28:    end if
29:  end for
30:   $r^* \leftarrow$  the first member minimizing  $\hat{V}(r)$ 
31:   $V^* \leftarrow \text{conn}_{r^*}(\beta(r^*))$ 
32:  for  $i = 1, \dots, M$  do
33:     $V^* \leftarrow V^* + \text{READ}(\text{D}(C, t), I_i, \beta_i(r^*))$ 
34:  end for
35:  return  $(r^*, V^*)$ 
36: end function
```

Definition 5.5. Let $\Pi_t(I, q)$ be the minimum of $\mathcal{B}_I(d_{t-1})(q)$ and the actual selected group costs, each extended from its selected representative by $L|q - r|_1$. Compute this minimum by Lemma 4.8 and whole-cell enumeration. A group's choice is cached for the whole cell, independent of which query first requests it. Lemma 4.8 shows that this local computation equals the global cone minimum for a good seed.

There is also an online action query at each scale passing Lemma 4.8. Put $r = r_b(q)$ and proceed only if this is an unskipped representative. Draw one action $\alpha_t \in \mathcal{C}_{b,r}$ by the rule just defined, which uses child histories with index less than t . Evaluate

$$L|q - r|_1 + \sum_i d_t(I_i, s_i(\alpha_t)).$$

Let $Y_t(I, q)$ be the minimum of these online query values, or $+\infty$ if there are none. At a nonleaf define

$$d_t(I, q) = \begin{cases} 0, & w_0(I, q) = 0, \\ \min \left\{ \Pi_t(I, q), \max \left\{ \frac{\Pi_t(I, q)}{1 + \delta}, Y_t(I, q) \right\} \right\}, & w_0(I, q) > 0. \end{cases}$$

Leaves continue to use their exact values.

At a positive w_0 state, this update truncates Y_t to the interval $[\Pi_t/(1 + \delta), \Pi_t]$: an inexpensive online action can improve the value only within this interval. The upper endpoint retains the Bellman comparison, while the lower endpoint transfers the spatial regularity of Π_t to d_t up to a factor $1 + \delta$. For a fixed pass, these definitions are well founded by induction on node height: current-index values are requested only at children, and at a fixed node the remaining refinement values have earlier indices.

Algorithm 21 One refinement entry

```
1: function REFINEMENTENTRY( $C, I, q, t$ )
2:   if  $I$  is a leaf then
3:     return LEAFCOST( $I, q$ )
4:   end if
5:    $(I_i)_{i=1}^M \leftarrow \text{CHILDREN}(I, M)$ 
6:    $w \leftarrow \text{READ}(\text{W}(C, 0), I, q)$ 
7:   if  $w = 0$  then
8:     return 0
9:   end if
10:   $\Pi \leftarrow \text{BELLMAN}(C, I, q, \text{D}(C, t - 1)); Y \leftarrow +\infty$ 
11:  for  $b \in \text{LOCALSCALES}(C, I, q)$  do
12:    for  $c_{\text{cell}} \in \text{LOCALCELLS}(b, q, w)$  do
13:       $G \leftarrow \text{GET}((\text{Group}, C, I, b, c_{\text{cell}}, t))$ 
14:      if  $G \neq \text{EMPTY}$  then
15:        Write  $G = (r^*, V^*)$ ;  $\Pi \leftarrow \min\{\Pi, V^* + L|q - r^*|_1\}$ 
16:      end if
17:    end for
18:     $g \leftarrow \max\{1, b/P\}$ ;  $r \leftarrow (g\lfloor q^-/g \rfloor, g\lfloor q^+/g \rfloor)$ 
19:    if ISREPRESENTATIVE( $C, I, b, r$ ) then
20:       $B_r \leftarrow \text{GET}((\text{Band}, C, I, b, r))$ 
21:      if  $B_r \neq \text{SKIPPED}$  then
22:         $\alpha \leftarrow \text{GET}((\text{Online}, C, I, q, b, t))$ 
23:         $v \leftarrow L|q - r|_1$ 
24:        for  $i = 1, \dots, M$  do
25:           $v \leftarrow v + \text{READ}(\text{D}(C, t), I_i, s_i(\alpha))$ 
26:        end for
27:         $Y \leftarrow \min\{Y, v\}$ 
28:      end if
29:    end if
30:  end for
31:  return  $\min\{\Pi, \max\{\Pi/(1 + \delta), Y\}\}$ 
32: end function
```

5.6 Deterministic invariants

Lemma 5.6. *For a good seed, fix the complete child tables and the choices of all groups at index t . For every parent state q , the local value $\Pi_t(I, q)$ equals the minimum of $w_0(I, q)$, all Bellman action cones, and all selected group cones. This is one fixed family of L -Lipschitz cones, independent of the requesting state.*

Proof. A group is keyed by the node, scale, whole cell, and index, together with the pass and copy identifiers. Its key omits q . Its representatives, samples, selected member, and actual selected cost do not change with the requesting state. Every selected member is an eligible representative, and its actual cost is nonnegative. Apply Lemma 4.8 to this fixed family and to the Bellman cones. The local search contains every competitive cone. Taking their global minimum with w_0 also proves the Lipschitz bound. $\square$

Lemma 5.7. *For a seed satisfying Definition 4.1, every outcome of the refinement obeys*

$$\begin{aligned} \max\{c/a, |z|\} &\leq d_t \leq d_{t-1} \leq w_0, \\ d_t(I, q) &\leq (1 + \delta)d_t(I, q') + L|q - q'|_1. \end{aligned}$$

At nonleaves one also has

$$\mathcal{B}_I(d_t)(q) \leq d_t(I, q) \leq \mathcal{B}_I(d_{t-1})(q).$$

For d_0 , the spatial bound holds with Lipschitz constant L and without the factor $1 + \delta$.

Proof. Induct on node height, and at a fixed node on t . Every extra candidate in Π_t is a current-value wide-action cone. The Bellman candidate uses earlier child values, which are at least the current ones by induction. Hence $\Pi_t \geq \mathcal{B}_I(d_t)$, whereas $\Pi_t \leq \mathcal{B}_I(d_{t-1})$. Each online candidate is a current-value wide-action cone too, so $Y_t \geq \mathcal{B}_I(d_t)$, including the empty case. Taking the specified minimum and maximum in Definition 5.5 proves the Bellman sandwich. In particular, no lower bound is inferred from $\Pi_t/(1 + \delta)$ alone.

At $t = 1$, Lemma 4.7 now gives $d_1 \leq d_0$. At later times the left inequality in the Bellman sandwich at time $t - 1$ gives $d_t \leq d_{t-1}$. The lower bound follows from Bellman preservation and the child induction.

Lemma 5.6 makes Π_t L -Lipschitz on every outcome of the group samples. Moreover $\Pi_t/(1 + \delta) \leq d_t \leq \Pi_t$. Thus

$$d_t(I, q) \leq \Pi_t(I, q) \leq \Pi_t(I, q') + L|q - q'|_1 \leq (1 + \delta)d_t(I, q') + L|q - q'|_1.$$

If $w_0(I, q) = 0$, the Bellman and Π_t values are zero, and the explicit zero branch agrees with these inequalities. The assertions for d_0 follow from the warmup. $\square$

For execution on an arbitrary seed, the local construction still gives nonnegative values at most its local w_0 . All representative lists and skipped-band tests remain finite. We use these unconditional facts for work bounds, and do not extend the correctness invariants of Lemma 5.7 to inaccurate seeds.

5.7 Accuracy of one group selection

For a band action, write $x_t(\alpha) = \sum_i d_t(I_i, s_i(\alpha))$.

Lemma 5.8. *For a good seed and an unskipped representative (b, r) , every band action $\alpha \in \mathcal{C}_{b,r}$ satisfies*

$$\begin{aligned} V_r(\beta; d_t) &\leq (1 + \delta)x_t(\alpha) + (6L + 2)\theta h_\Sigma, \\ d_t(I_i, s_i(\beta)) &\leq 2h_i. \end{aligned}$$

Proof. Two band actions differ by at most $4\theta b/M$ in each child's endpoint metric. Rounding the canonical action moves its two coordinates by less than $2\theta h_i$ in total. Since $h_i \geq b/M$, the distance from any band state to its rounded state is at most $6\theta h_i$. The canonical action has shared endpoints, so its rounded version has $\text{conn}_r(\beta) \leq 2\theta h_\Sigma$. The spatial bound in Lemma 5.7 now proves the first inequality. For the second, $d_t \leq w_0$ and the L_0 -Lipschitz bound give $w_0(I_i, s_i(\beta)) \leq h_i + 2L_0\theta h_i \leq 2h_i$. $\square$

Lemma 5.9. *Under the seed hypothesis of Definition 4.1, fix complete child tables and one group at time t . With conditional probability at least $1 - H^{-4}$, all action-cost estimates used to choose its representative are within γb of their actual costs. Consequently, for any specified unskipped member (b, r) of this group, on the same event, simultaneously for every valid parent state q and every $\alpha \in \mathcal{C}_{b,r}$,*

$$\Pi_t(I, q) \leq L|q - r|_1 + (1 + \delta)x_t(\alpha) + e_*b, \quad e_* = 2\gamma + 512LF\theta \leq \kappa\delta.$$

Here the inequality uses the global cone minimum, equivalently its local evaluation from Lemma 4.8.

Proof. Conditioning fixes the bands, the h_i , the rounded actions, and all their child costs. The parent's group samples remain independent of these data. For a fixed member and sampled scale h , normalize one sample to

$$X = \frac{M}{b} \mathbf{1}_{h_i=h} d_i(I_i, s_i(\beta)),$$

where i is uniform in $\{1, \dots, M\}$. By Lemma 5.8,

$$0 \leq X \leq 2Mh/b, \quad \mathbb{E}X \leq 2h_\Sigma/b \leq 128F, \quad \text{Var}(X) \leq 256FMh/b.$$

When $k_h < M$, the number of independent samples is at least Q_1Mh/b . Bernstein's inequality [27], with normalized error $\alpha_0 = \gamma/(2\ell)$, gives a failure probability at most

$$2 \exp\left(-\frac{Q_1\alpha_0^2}{512F + 4\alpha_0/3}\right) \leq 2 \exp(-\ell^{37}/3000).$$

When $k_h = M$, the contribution is exact and has no failure event. There are at most 2ℓ scales and at most $(P+1)^2$ group members. A union bound is at most H^{-4} at the absolute cutoff. Summing the errors over scales gives error at most γb for every member. Shared samples between members cause no difficulty for this union bound.

On this event, estimated-cost minimization followed by actual evaluation selects a cost at most that of the specified member plus $2\gamma b$. The selected representative is in the same cell, so its endpoint-pair distance from r is at most $2\theta b$. Using Lemma 5.8 and $h_\Sigma \leq 64Fb$ gives the stated group bound, since its additive coefficient is at most

$$2\gamma + (384LF + 128F + 2L)\theta \leq 2\gamma + 512LF\theta \leq \kappa\delta.$$

The comparison with every band action is simultaneous because Lemma 5.8 is deterministic and uniform in that action. $\square$

This is a conditional statement for one whole cell and time. The accuracy proof will not require all group samples throughout the recursion to succeed together. Instead it will charge each group's failure probability inside an expected regret bound at its comparison node.

6 A comparison tree and cost-weighted curvature

Multiscale restrictions of alignment paths, controlled by bounds on their total deviation, are central to Mao and Rubinstein's approximation scheme [21]. Here the weights are warmup cost estimates. They need not add exactly from children to parent, so our energy calculation also charges their mass discrepancy and the difference between a comparison action and a minimum wide action.

Fix a good seed as in Definition 4.1. We construct a comparison tree from exact alignments. Its states are fixed before the randomness of this refinement is drawn. The warmup values supply weights on this tree. A telescoping energy calculation controls the total cost of nodes with large curvature or a large gap between the comparison action and the minimum wide action. At the other nodes, the velocity test has two useful outcomes. Passing it places the comparison action in the computed band; failing it means that length imbalance nearly accounts for the warmup cost. These alternatives distinguish nodes with a useful band from nodes whose error is bounded by their length imbalance.

6.1 Exact paths and rounded child states

Start at a state (I, q) of true cost $D = c(I, q)$. If $D = 0$, the good-seed bounds force all its estimates to be zero, so assume $D > 0$. Stop a branch at a leaf of the interval tree or at a state of cost zero. At each remaining node, write $c = c(I, q) > 0$, choose the dyadic scale b with $b \leq c < 2b$, and put

$$r = r_b(q), \quad e = |q - r|_1 \leq 2b/P.$$

Choose an optimal alignment for (I, r) and points on this path at the child cuts, in their path order. Use its exact endpoints at the outside cuts. Repeated cuts may use the same path point. A vertical segment can also be assigned to one of the intervening empty children. Round the internal target anchors down to the grid at scale b . The resulting child states $q_1, \dots, q_M$ remain ordered and share endpoints, with outside endpoints exactly r . Denote this action by α^* , and continue the construction at its child states.

Fix a deterministic order for all choices of optimal paths and cut points. The entire comparison tree, including its stopping states, then depends only on the input strings and its starting state. In particular it is independent of every group sample and online draw in the refinement. This tree is used only in the proof.

Lemma 6.1. *At each nonterminal of the comparison tree, (b, r) is a representative, b satisfies the local scale restriction in Lemma 4.8, and α^* is a wide action whose endpoint shifts are within $6b$ of $p_0 = r^- - l$. Writing $c_i = c(I_i, q_i)$, we have*

$$c(I, r) \leq \sum_i c_i \leq (1 + 4M/P)c, \quad c \leq Le + \sum_i c_i.$$

Let C_{term} be the sum of terminal true costs. Then

$$\begin{aligned} C_{\text{term}} &\leq (1 + 8MJ/P)D \leq 2D, \\ \sum_{\text{nonterminal}} c &\leq 2JD, \quad \sum_{\text{nonterminal}} e \leq 4JD/P. \end{aligned}$$

The sum of true costs at any depth is at most $2D$, also if stopped costs are carried unchanged to subsequent depths. At every terminal, $m = c$ and every refinement estimate equals c .

Proof. If the grid spacing is one, no rounding changes any endpoint. Otherwise an endpoint moves by less than b/P . Thus

$$c - e \leq c(I, r) \leq c + e.$$

The initial-cone bounds give

$$w_0(I, r) \geq c(I, r)/a \geq b/(4a), \quad w_0(I, r) \leq 2Fc(I, r) \leq 8Fb,$$

so (b, r) is a representative. At q , the same bounds imply $w_0(I, q)/(16F) \leq b \leq 8aw_0(I, q)$, as required for Lemma 4.8.

Along an alignment, each change of shift is caused by an insertion or deletion and is at most the cost of that part of the path. Hence every unrounded anchor shift differs from p_0 by at most $c(I, r)$. Rounding adds less than b/P , so the distance is at most

$$c + e + b/P < (2 + 3/P)b < 6b.$$

This proves the wide-action and radius assertions.

The exact subpath costs before rounding sum to $c(I, r)$. Each internal anchor affects the endpoints of two child problems. Endpoint Lipschitz continuity gives

$$\sum_i c_i \leq c(I, r) + 2(M-1)b/P \leq c + 2Mb/P \leq (1 + 4M/P)c.$$

Concatenation gives the other inequality in the first part of Lemma 6.1; it also gives $c \leq c(I, r) + e \leq \sum_i c_i + Le$. These arguments apply without change to repeated cuts and empty child intervals.

At each successive frontier of the tree, carry terminal costs unchanged. The frontier sum grows by a factor at most $1 + 4M/P$ in one step. The parameter bounds give $4MJ/P \leq 1/2$, and consequently

$$(1 + 4M/P)^J \leq 1 + 8MJ/P \leq 2.$$

This proves the terminal and level bounds. There are at most J nonterminal levels. Summing $e \leq 2b/P \leq 2c/P$ proves the remaining bounds in Lemma 6.1.

At a leaf, every table uses the exact answer. At a zero-cost state, the good seed is zero and the upper bound $w_0 \leq 2Fc$ forces all warmup and refinement values to be zero. These are precisely the terminal cases. $\square$

6.2 Mass and energy telescopes

If the warmup masses added exactly from children to parent, the energies z^2/m would telescope up to endpoint-rounding errors. The masses do not add exactly: a parent's mass can be either larger or smaller than its children's total. The warmup inequalities bound the total upward discrepancy, and conservation of mass over the comparison tree then bounds the total downward discrepancy as well. This is why the energy argument can use cost estimates instead of exact additive weights.

At a nonterminal use the representative and child states just constructed, and define

$$m_q = m(I, q), \quad m_i = m(I_i, q_i), \quad W = \sum_i m_i, \quad G = W - Z, \quad R = m_q - W.$$

Here Z is the minimum wide-action cost with child table m , as in Section 5. The comparison action has zero connection cost, so $W \geq Z$. The lower invariants and the comparison costs imply

$$0 < \max\{c(I, r)/a, |z(I, r)|\} \leq Z \leq W \leq 4Fc < 8Fb.$$

In particular, the denominator Z defining $\hat{v}$ is positive at every nonterminal comparison representative.

Write $z_i = z(I_i, q_i)$ and $z_r = z(I, r)$. Since the child targets share endpoints, $\sum_i z_i = z_r$. Define the curvature quantity

$$K = \sum_i \frac{z_i^2}{m_i} - \frac{z_r^2}{W},$$

with $0/0 = 0$. A zero m_i necessarily has $z_i = 0$ by the lower invariant. For $v = z_r/W$, the weighted variance identity is

$$K = \sum_{i:m_i>0} \frac{(z_i - vm_i)^2}{m_i} \geq 0.$$

These weights are costs supplied by the warmup. Define the energy of a comparison state by $E(I, q) = z(I, q)^2/m(I, q)$, again with $0/0 = 0$. Figure 3 shows how the local energy differences sum over the comparison tree.

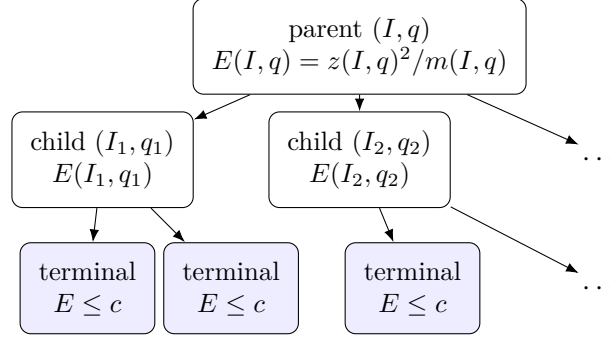


Figure 3: Energy cancellation on the comparison tree. At a nonterminal, $K \leq \sum_i E(I_i, q_i) - E(I, q) + 3e + 2 \max\{-R, 0\}$. Summing cancels each internal energy once with a positive sign and once with a negative sign. Only terminal energies and the negative root energy remain. The tree and its exact comparison paths are analysis objects.

Lemma 6.2. *On the fixed comparison tree, put $h = Le + w_0(I, q)/T$ at each nonterminal. Then*

$$\sum_{\text{nonterminal}} (K + G) \leq 4C_{\text{term}} + 3 \sum_{\text{nonterminal}} e + 4 \sum_{\text{nonterminal}} h.$$

The averaging errors also satisfy

$$\sum_{\text{nonterminal}} h \leq 512FJD/P + 4FJD/T.$$

Consequently,

$$\sum_{\text{nonterminal}} (K + G) \leq 5000F(1 + J/P + J/T)D.$$

Proof. First control the mass discrepancy. Apply the averaging inequality of Lemma 4.7 to the comparison representative. It gives

$$m_q - \frac{w_0(I, q) - w_T(I, q)}{T} \leq Le + Z.$$

Thus

$$R + G \leq h, \quad R_+ \leq h, \quad G \leq R_- + h, \quad R_{\pm} = \max\{\pm R, 0\}.$$

The masses cancel exactly across the tree:

$$\sum R = m_{\text{root}} - C_{\text{term}}, \quad \sum R_- \leq \sum R_+ + C_{\text{term}} \leq \sum h + C_{\text{term}}.$$

In particular, summing $G \leq R_- + h$ gives $\sum G \leq C_{\text{term}} + 2 \sum h$.

Next compare the parent and child energies. At a nonterminal put $z_q = z(I, q)$. We have $|z_q| \leq m_q$, $|z_r - z_q| \leq e$, and $|z_r| \leq W$. Moreover,

$$e \leq 2c/P \leq 2am_q/P \leq m_q,$$

because $a \leq A_{\text{init}}$ and $2A_{\text{init}} < P$. $|z_r| \leq 2m_q$. The identity

$$\frac{z_q^2}{m_q} - \frac{z_r^2}{W} = \frac{z_q^2 - z_r^2}{m_q} + \frac{z_r^2(W - m_q)}{m_q W}$$

has first term at most $3e$. If $W \leq m_q$, its second term is nonpositive; otherwise the bounds $|z_r| \leq W$ and $|z_r| \leq 2m_q$ bound it by $2(W - m_q) = 2R_-$. It follows that

$$K \leq \sum_i E(I_i, q_i) - E(I, q) + 3e + 2R_-.$$

No zero denominator occurs at a nonterminal, by the mass bounds; zero child masses have already been treated.

Terminal energies are at most their true costs, and the root energy is nonnegative. Summing the local energy inequality gives

$$\sum K \leq C_{\text{term}} + 3 \sum e + 2 \sum R_- \leq 3C_{\text{term}} + 3 \sum e + 2 \sum h.$$

Adding the preceding bound on $\sum G$ proves the first claimed bound. It remains only to substitute the comparison-tree bounds. By Lemma 6.1 and $w_0 \leq 2Fc$,

$$\sum h \leq 512FJD/P + 4FJD/T.$$

Together with Lemma 6.1, this yields

$$\sum (K + G) \leq 8D + (2048F + 12)JD/P + 16FJD/T,$$

which implies the total curvature bound. $\square$

Call a nonterminal *bad* if $K > bH^{-1/4}$ or $G > bH^{-1/4}$; otherwise call it *non-bad*. These are conditions on the deterministic comparison tree and its warmup values.

Corollary 6.3. *The total true cost of bad nodes satisfies*

$$\sum_{\text{bad}} c \leq 10000FH^{1/4}(1 + J/P + J/T)D.$$

Proof. At a bad node, $c < 2b < 2H^{1/4}(K + G)$. Sum and apply Lemma 6.2. $\square$

6.3 Coverage by the computed bands

The total-curvature bound controls the cost of bad nodes. At every other node that passes the velocity test, the comparison action survives the band and rounding guards. Small curvature is useful here because it keeps each prefix shift close to the shift predicted from its cost.

Lemma 6.4. *At a non-bad comparison node with $|\hat{v}| \leq 1 - \tau$, the action α^* belongs to $\mathcal{C}_{b,r}$. Its representative is not skipped: the band is nonempty, $h_\Sigma \leq 64Fb$, and the rounded action $\beta(b, r)$ passes both the wide-action and imbalance guards.*

Proof. Let p_j^* be the comparison shift at internal cut j , and write $W_j = \sum_{i \leq j} m_i$. The comparison prefix is feasible for $\Phi_j(p_j^*)$, so this prefix cost is at most W_j . Conversely, concatenate any feasible prefix in that minimum with the comparison suffix. The prefix's final jump is exactly its connection to the suffix. The resulting wide action has cost at least Z , and hence

$$W_j - G \leq \Phi_j(p_j^*) \leq W_j.$$

With $v = z_r/W$, we also have

$$|v - \hat{v}|W = |z_r|G/Z \leq G.$$

Weighted Cauchy–Schwarz in the variance identity for K gives

$$\sum_i |z_i - vm_i| \leq \sqrt{KW}.$$

The shared endpoints imply $p_j^* - p_0 = \sum_{i \leq j} z_i$. Combining the last three observations yields

$$|p_j^* - p_0 - \hat{v}\Phi_j(p_j^*)| \leq \sqrt{KW} + 2G \leq \tau\theta b/M.$$

For completeness, the last numerical inequality follows from $K, G \leq bH^{-1/4}$, $W < 8Fb$, and the parameter bounds:

$$\frac{M}{b}(\sqrt{KW} + 2G) \leq \sqrt{8\ell} H^{-3/40} + 2H^{-1/5} \leq \ell^{-44} = \tau\theta.$$

Indeed $\ell^{200} \leq H^{1/100}$ bounds the middle expression by $\sqrt{8\ell} \ell^{-1500} + 2\ell^{-4000}$. The radius and shared-endpoint conditions were proved in Lemma 6.1. Thus the prefix-shift bound proves $\alpha^* \in \mathcal{C}_{b,r}$.

The diameter bound for the band implies that any of its child states is within $4\theta b/M$ of q_i in endpoint metric. Using the L_0 -Lipschitz property of w_0 and the factor of at most two from dyadic ceiling, we get

$$\begin{aligned} h_\Sigma &\leq 2b + 2 \sum_i w_0(I_i, q_i) + 8L_0\theta b \\ &\leq 2b + 4F \sum_i c_i + 8L_0\theta b \leq 64Fb. \end{aligned}$$

The last step uses Lemma 6.1, $c < 2b$, and the parameter range. This verifies the sum guard.

Under the fixed good seed, the rounding verification following the rounded-action guard in Definition 5.1 applies as soon as this sum guard holds. It shows that $\beta(b, r)$ is a wide action and satisfies the imbalance guard. Rounding preserves the order of each child's two endpoints; different child spacings may separate shared endpoints, with total connection charge at most $2\theta h_\Sigma$. Thus every guard passes. $\square$

7 Conditional regret and the first crossing

Fix a good seed as in Definition 4.1, and fix the comparison tree from Section 6. We will bound the error accumulated at its nodes over the refinement indices. The estimates may lie below true cost when $a > 1$. To keep such underestimation from cancelling an error elsewhere, define, for the analysis only,

$$\bar{d}_t(I, q) = \max\{c(I, q), d_t(I, q)\}.$$

Definition 7.1. At a nonterminal comparison node use the notation $c, b, r, e, q_i, m_i, W, G, K$ of Section 6, and set

$$\mathcal{R}_t(I, q) = \bar{d}_t(I, q) - Le - \sum_i \bar{d}_t(I_i, q_i).$$

These quantities telescope over the fixed comparison tree. We first bound their sum in time at an arbitrary node, then improve that bound on a node whose comparison action belongs to a computed band.

7.1 A telescoping bound at every comparison node

Lemma 7.2. *At every nonterminal comparison node,*

$$\sum_{t=1}^S \mathcal{R}_t \leq \sum_i (\bar{d}_0 - \bar{d}_S)(I_i, q_i) \leq \sum_i (m_i - |z_i|) \leq W - |z(I, r)|,$$

where $z_i = z(I_i, q_i)$. In particular, the bound is at most $4Fc$. If the node is not bad and $|\hat{v}| > 1 - \tau$, it is at most $(H^{-1/4} + 4\tau F)c$.

Proof. The Bellman option and the comparison action give

$$d_t(I, q) \leq Le + \sum_i d_{t-1}(I_i, q_i).$$

The right side with all child values replaced by their clipped values is at least c , by Lemma 6.1. It also bounds $\bar{d}_t(I, q)$. Subtracting the same expression at index t and summing proves the first inequality in the displayed regret bound.

For the second inequality, consider a child separately. If $d_0 \leq c_i$, monotonicity makes both clipped endpoint values equal to c_i , so their difference is zero, while $m_i \geq |z_i|$. If $d_0 > c_i$, then $\bar{d}_0 = d_0 \leq m_i$ and $\bar{d}_S \geq |z_i|$. This proves the required difference bound in both cases; it does not require $\bar{d}_0 \leq m_i$ in the first case. Finally, $\sum_i |z_i| \geq |z(I, r)|$ and $W \leq 4Fc$.

For the last assertion, $|\hat{v}| = |z(I, r)|/Z > 1 - \tau$ implies

$$W - |z(I, r)| \leq G + \tau W \leq (H^{-1/4} + 4\tau F)c,$$

using $G \leq bH^{-1/4}$ and $b \leq c$ at a node that is not bad. $\square$

7.2 Regret within a predicted band

Lemma 7.3. *Fix a good seed and an unskipped band $\mathcal{C} = \mathcal{C}_{b,r}$. Condition on the complete child tables, including all indices through S . For every fixed action $\alpha^* \in \mathcal{C}$ and every $0 \leq s \leq S$,*

$$\begin{aligned} \mathbb{E} \left[\sum_{t=1}^s \{x_t(\alpha_t) + \kappa(x_{t-1}(\alpha_t) - x_t(\alpha_t))\} \middle| \text{child tables} \right] &\leq \sum_{t=1}^s x_t(\alpha^*) \\ &\quad + h_\Sigma \left(\frac{\ln d_{\mathcal{C}}}{\eta} + 2\kappa + sH^{-3} \right). \end{aligned}$$

Proof. Apply Lemma 5.2 to an unskipped band $\mathcal{C} = \mathcal{C}_{b,r}$, conditioning on the complete child tables, including the warmup and all indices through S . The band, the scales h_i , and its action hull $\mathcal{Q}$ are then fixed. Take the child coordinates as blocks and set

$$f_t(i, s) = d_t(I_i, s)/h_i.$$

The action hull is compact and convex, and averaging its action vectors gives a positive feasible vector. Every vector has mass h_i/h_Σ in child block i . The refinement invariants give $0 \leq d_S \leq \dots \leq d_0 \leq w_0 \leq h_i$ on band coordinates, so the normalized costs meet the monotonicity and unit bounds of the lemma.

Two states in a child block have endpoint-pair distance at most $4\theta b/M$. The spatial bound in Lemma 5.7, used in both directions and with $d_t \leq h_i$, bounds their cost difference at one index by

$\delta h_i + 4L\theta b/M$; the stronger bound at index zero also suffices. Since $h_i \geq b/M$, the within-block range of Δ_t is at most

$$2\delta + 8L\theta \leq 3\delta \leq \kappa/\eta.$$

Here $8L\theta = 1024\ell^{-39} \leq \ell^{-10}$ and $3\eta\delta = 3\ell^{-5} \leq \kappa$. The lemma applies to exactly the objective in Definition 5.4, with $d = d_C$. Conditioning on child tables leaves the parent group samples and online draws with their prescribed laws, since their streams are separate from descendant streams. No independence among child tables is needed.

The fixed coordinate weights give

$$\begin{aligned} h_\Sigma \langle f_t, \rho(\alpha) \rangle &= x_t(\alpha), \\ h_\Sigma \langle f_t + \kappa \Delta_t, \rho(\alpha) \rangle &= x_t(\alpha) + \kappa(x_{t-1}(\alpha) - x_t(\alpha)). \end{aligned}$$

If α_t is drawn from the computed mixture, its expected action vector is that mixture's mean. Lemma 5.2, with $\zeta = H^{-3}$, gives the stated conditional bound for every fixed band action and prefix. $\square$

The decrement term in this bound pays for the clipping in the parent update.

7.3 Clipping before and after the first crossing

We now apply the entropy inequality at a comparison node that is not bad and satisfies $|\hat{v}| \leq 1 - \tau$. Lemma 6.4 ensures that its comparison action α^* belongs to the unskipped band $\mathcal{C}_{b,r}$ and that all its guards pass. Write

$$A_t(\alpha) = Le + x_t(\alpha).$$

The entropy bound controls these action costs. The following proposition transfers that control to the clipped parent regret.

Proposition 7.4. *At a non-bad comparison node passing $|\hat{v}| \leq 1 - \tau$, condition on the complete child tables. Then*

$$\begin{aligned} \mathbb{E} \left[\sum_{t=1}^S \mathcal{R}_t \mid \text{child tables} \right] &\leq h_\Sigma \left(\frac{\ln d_C}{\eta} + 2\kappa + SH^{-3} \right) \\ &\quad + 2\delta c + 2SH^{-4}Fc. \end{aligned}$$

Here $h_\Sigma \leq 64Fb$ and $\ln d_C \leq 4(p_* + 21)\ell$.

Proof. We split the indices when the least band-action cost first falls below c . Before that crossing, a decrease in child cost pays for the clipping excess. The crossing contributes one additional charge, and every later regret is nonpositive.

Let $t_\times$ be the first $t \in \{1, \dots, S\}$ for which $\min_{\alpha \in \mathcal{C}} A_t(\alpha) < c$, or put $t_\times = S + 1$ if there is none. This index depends only on the conditioned child tables. It is an analysis device, not a test performed by the algorithm. We will use Lemma 7.3 on the fixed prefix $1 \leq t < t_\times$.

Let E_t be the failure event for the group approximation associated with this representative. By Lemma 5.9, $\Pr(E_t \mid \text{child tables}) \leq H^{-4}$, and on its complement the stated group bound holds for every action in the band. For the online action α_t sampled for this band, we claim that on the prefix

$$\begin{aligned} \bar{d}_t(I, q) &\leq Le + x_t(\alpha_t) + \kappa(x_{t-1}(\alpha_t) - x_t(\alpha_t)) \\ &\quad + 2Fc \mathbf{1}_{E_t}. \end{aligned}$$

Every action cost $A_t(\alpha)$ is at least c on this prefix, and the online candidate satisfies $Y_t(q) \leq A_t(\alpha_t)$. If $\bar{d}_t(I, q) \leq A_t(\alpha_t)$, the claim follows at once from child monotonicity. Otherwise, the update in Definition 5.5 forces

$$\Pi_t(q) > (1 + \delta)A_t(\alpha_t), \quad d_t(I, q) = \frac{\Pi_t(q)}{1 + \delta}.$$

The earlier-index Bellman option gives $\Pi_t(q) \leq Le + x_{t-1}(\alpha_t)$. Write $\Delta x = x_{t-1}(\alpha_t) - x_t(\alpha_t)$. Combining the two bounds gives

$$\Delta x > \delta A_t(\alpha_t) \geq \delta c \geq \delta b.$$

On E_t^c , the stated group bound and $e_* \leq \kappa\delta$ now give the full excess calculation

$$\begin{aligned} d_t(I, q) - A_t(\alpha_t) &\leq \frac{e_*b - \delta Le}{1 + \delta} \leq \frac{e_*b}{1 + \delta} \\ &\leq \frac{\kappa\delta b}{1 + \delta} \leq \kappa\Delta x. \end{aligned}$$

On E_t , the added term $2Fc$ suffices because $\bar{d}_t(I, q) \leq 2Fc$. This proves the prefix clipping bound without requiring independence between the sampled action and the group event.

If $t_\times \leq S$, choose an action attaining the crossing. On $E_{t_\times}^c$, its group bound gives

$$\Pi_{t_\times}(q) \leq Le + (1 + \delta)x_{t_\times}(\alpha) + e_*b < c + \delta c + e_*b \leq c + 2\delta c.$$

Thus the clipped regret at that index is at most $2\delta c$, with an additional $2Fc$ allowed on failure. At every later index, the same crossing action and child monotonicity imply $\mathcal{B}_I(d_{t-1})(q) < c$. Hence $d_t(I, q) < c$, $\bar{d}_t(I, q) = c$, and the regret is nonpositive by Lemma 6.1.

Sum the prefix clipping bound. Apply Lemma 7.3 with $s = t_\times - 1$ and the comparison action α^* . Since $x_t(\alpha^*) \leq \sum_i \bar{d}_t(I_i, q_i)$, its comparator costs are bounded by the child terms subtracted in each regret. The expected prefix regret is at most

$$h_\Sigma \left(\frac{\ln d_C}{\eta} + 2\kappa + SH^{-3} \right),$$

apart from the explicit failure charges in the prefix clipping bound. The prefix and possible crossing contain at most S failure events, each of conditional probability at most H^{-4} . Adding their expected costs and the crossing charge proves Proposition 7.4.

Finally each child has at most $(21P)^2$ possible states within the wide shift radius. Thus $d_C \leq M(21P)^2$, from which $\ln d_C \leq 4(p_* + 21)\ell$ follows using $\log_2 H \leq \ell$. The bound on h_Σ is one of the band guards. $\square$

The expectation in Proposition 7.4 concerns one node after fixing its child tables. Integrating it and summing over the fixed comparison tree uses only linearity of expectation. No event asserting that every group in the recursive computation succeeds is required.

8 Accuracy and the sequence of seed passes

We first sum the node estimates to prove the refinement guarantee. We then use that guarantee to improve the simultaneous seed factor and obtain the answer to the original input. Throughout the first step the entire seed table is fixed and good in the sense of Definition 4.1.

8.1 The refinement guarantee

Theorem 8.1. *Fix a good seed U as in Definition 4.1, and use the refinement in Definition 5.5. Its table d_S is finitely executable and satisfies, for every state (I, q) ,*

$$d_S(I, q) \geq c(I, q)/a, \quad \mathbb{E}[(d_S(I, q) - c(I, q))_+] \leq c(I, q)/\ell.$$

The first inequality holds for every outcome of the refinement randomness; the expectation in the second is conditional on the fixed table U . At zero-cost states the returned value is zero for every outcome.

Proof. Fix a starting state with true cost $D > 0$ and its comparison tree from Section 6. For brevity write $\bar{d}_t = \max(c, d_t)$ at every state, as in Section 7. Terminal values are exact, so summing Definition 7.1 over the tree at a fixed index gives the identity

$$\bar{d}_t(\text{root}) = C_{\text{term}} + L \sum_{\text{nonterminal}} e + \sum_{\text{nonterminal}} \mathcal{R}_t.$$

Average this identity over $t = 1, \dots, S$. Since the root estimates decrease with t , the final positive error satisfies the deterministic bound

$$(d_S(\text{root}) - D)_+ \leq C_{\text{term}} - D + L \sum_{\text{nonterminal}} e + \frac{1}{S} \sum_{\text{nonterminal}} \sum_{t=1}^S \mathcal{R}_t.$$

Lemma 6.1 bounds the first two terms by $(8MJ/P + 4LJ/P)D$. We bound the last sum using the three classes of comparison nodes.

On bad nodes use Lemma 7.2 and $W \leq 4Fc$. Corollary 6.3 bounds their total contribution by

$$40000F^2H^{1/4}(1 + J/P + J/T)D.$$

On nodes that are not bad but satisfy $|\hat{v}| > 1 - \tau$, the same lemma gives $(H^{-1/4} + 4\tau F)c$. Every other nonterminal satisfies Lemma 6.4, so Proposition 7.4 applies. Its dimension and mass bounds are

$$\ln d_C \leq 4(p_* + 21)\ell, \quad h_\Sigma \leq 64Fb \leq 64Fc.$$

The cost sum over all nonterminal levels is at most $2JD$.

The conditional estimate at a node is valid after fixing its complete child tables. Taking expectations and then summing uses only the tower property and linearity of expectation. Independence between comparison nodes, or simultaneous success of their group samples, is unnecessary. Combining these bounds, the expected excess at the starting state divided by D is at most

$$\begin{aligned} \mathcal{E} &= (8M + 512F)J/P + 40000F^2H^{1/4}(1 + J/P + J/T)/S \\ &\quad + \frac{2J}{S} \left[H^{-1/4} + 4\tau F + 64F \left(\frac{4(p_* + 21)\ell}{\eta} + 2\kappa \right) + 2\delta \right] + 132FJH^{-3}. \end{aligned}$$

Here the last term includes both numerical-mixture loss and group-failure loss: they are at most $128FJH^{-3}$ and $4FJH^{-4}$, respectively.

We verify the numerical bound explicitly. Substituting $J \leq 100H/\ell$, $T \geq H^{3/4}$, $S = H/\ell^2$, $F = \ell$, $M \leq H^{1/20}$ and $P \geq H^{20}$ yields

$$\begin{aligned} \mathcal{E} &\leq 52000H^{-18} + 8.04 \cdot 10^6 \ell^4 H^{-1/2} + 200\ell H^{-1/4} \\ &\quad + \frac{10^6(p_* + 21)}{\ell^2} + 13200H^{-2}. \end{aligned}$$

For the fourth term we have included $[26400 + 51200(p_* + 21)]/\ell^2 + 400/\ell^9$ from the bracket in the bound on $\mathcal{E}$. Each of the five displayed terms is at most $1/(8\ell)$ at $\ell \geq 2^{1000}$ and $H \geq 2^{\ell/2}$. For example, the second requires only

$$\log_2(8 \cdot 8.04 \cdot 10^6) + 5 \log_2 \ell \leq \ell/4;$$

it holds at the cutoff and remains true thereafter. The other negative-power terms require weaker inequalities of the same form. For the fourth, use $p_* + 21 \leq \sqrt{\ell}$ and $\sqrt{\ell} \geq 2^{500} > 8 \cdot 10^6$. Consequently $\mathcal{E} < 1/\ell$.

Since $\bar{d}_S - D = (d_S - D)_+$, this is the required expected positive-excess bound. The deterministic lower bound is the invariant from Section 5. When $D = 0$, that invariant and $w_0 \leq 2Fc$ give $d_S = 0$ directly, without dividing by D . The mixtures are supplied by Lemma 5.3. All local searches are finite, and each recursive table dependency decreases the node height or the table index. The exact sampler has finite expected cost. Together these facts give finite executability. $\square$

8.2 Improving simultaneous seeds

The initial table of Theorem 3.5 has factor A_{init} . The algorithm maintains a power-of-two factor A and an integer seed $U \in [0, N]$ on the full state domain. Goodness is the event in Definition 4.1; the algorithm does not test it.

While $A > F$, set $a = A/F$ and use H^2 independent copies of the refinement, conditional on the previous complete table. For each state, form the H^2 integers $\lceil ad_S \rceil$, take their upper middle order statistic, and clamp the result to $[0, N]$. This defines the next seed. Update

$$A \leftarrow 4a.$$

All copies share the preceding table, but use independent new streams. Each copy defines a random table with its own independent streams. Entries of the copies and of the next seed are stored when requested.

The seed descriptor U_s stores its pass number s and the immutable records of its H^2 copies. Each record retains the preceding seed. Creating these records allocates no table entries. In a cache key, U_s denotes the pass number; its copy records are stored once.

Algorithm 22 A lazy median seed entry

```

1: function SEEDENTRY( $U_s, I, q$ )
2:   for  $j = 1, \dots, H^2$  do
3:      $C \leftarrow U_s.\text{copies}[j]$ 
4:      $v_j \leftarrow \lceil C.a \text{ READ}(D(C, S), I, q) \rceil$ 
5:   end for
6:   Select the  $(H^2/2 + 1)$ -st smallest  $v_j$ , denoted  $v_{\text{med}}$ 
7:   return  $\min\{N, \max\{0, v_{\text{med}}\}\}$ 
8: end function

```

Lemma 8.2. *Conditional on any fixed good preceding seed, the next seed is good for factor $4a$ except with probability at most $\exp(10H - H^2)$.*

Proof. At a fixed state of positive integer cost c , the refinement theorem and Markov's inequality give

$$\Pr[d_S > 2c] \leq 1/\ell.$$

Every copy has $\lceil ad_S \rceil \geq c$. A copy with $d_S \leq 2c$ has $\lceil ad_S \rceil \leq 2ac$, because a and c are integers. If strictly more than half the copies have this upper bound, their upper middle order statistic does too, hence in particular is at most $4ac$. The probability that at least half fail is at most

$$2^{H^2} \ell^{-H^2/2} \leq e^{-H^2}.$$

Clamping cannot spoil the lower bound $c \leq N$ or the upper bound. At zero-cost states every copy is zero under the good seed.

There are at most $2M \max(1, |x|)$ labeled nodes and at most $(|y| + 1)^2$ valid states per node. Their product is less than e^{10H} in the large branch. Union over this full domain proves the assertion. No independence between different states is used. $\square$

Proposition 8.3. *The sequence of median passes, followed by one final refinement, uses*

$$s_{\text{pass}} = O\left(\frac{(p_* + 1)\ell}{\log_2 \ell}\right) < H$$

passes. The probability that any seed used by these passes is not good is less than $0.01 + H \exp(10H - H^2) < 1/10$.

Proof. The update preserves powers of two and strictly decreases A , since $F > 4$. Let s_{pass} include the median passes and one final refinement. Then

$$s_{\text{pass}} = O\left(\frac{(p_* + 1)\ell}{\log_2 \ell}\right) < H.$$

Indeed each median pass reduces $\log_2 A$ by $\log_2 F - 2$, starting from $(p_* + 4) \log_2 H$ and stopping by $A \leq F$. Apply Lemma 8.2 to the first failed transition after a good history. The total probability that any seed needed by the algorithm is not good is less than

$$0.01 + H \exp(10H - H^2) < 1/10.$$

The initial 0.01 is the simultaneous initial-table failure probability. This estimate covers adaptive requests because every seed guarantee is over its entire potential domain. $\square$

8.3 The final answer

When $A \leq F$, use one new refinement with $a = 1$ at the full root state $(I, q) = ((0, |x|), (0, |y|))$. Clamp its value to $[0, N]$ and round down to obtain $\hat{D}$. Fix any complete good seed before exposing this final refinement's independent randomness. For the positive cost $D = \text{ED}(x, y)$ the refinement theorem gives the deterministic lower bound $d_S \geq D$ under this conditioning and

$$\Pr[d_S > (1 + \varepsilon)D \mid U \text{ fixed and good}] \leq \frac{1}{\ell^\varepsilon} \leq \frac{1}{10}.$$

Together with Proposition 8.3, the unconditional failure probability is less than $1/5$, and thus less than $1/3$. The lower bound is not asserted on executions with a bad seed.

Clamping preserves the lower bound because $D \leq N$. Rounding down also preserves it because D is an integer. Neither operation worsens the upper bound. The initial equality test returns zero deterministically when $D = 0$, and the exact branches of Section 2 are also correct. This establishes the accuracy assertion of Theorem 1.1 on every input. It remains to prove that the finite

computations just specified, with all randomness and arithmetic charged, have the claimed expected running time.

The driver initializes the parameters and shared data, creates the seed and copy records, and requests the final root value. Its table requests use the procedures already specified.

Algorithm 23 The complete approximation scheme

Input: Explicit strings x, y and canonical rational input $\varepsilon \in (0, 1)$

Output: An integer $(1 + \varepsilon)$ estimate with probability at least $2/3$

```

1: function APPROXIMATEED( $x, y, \varepsilon$ )
2:   if  $x = y$  then
3:     return 0
4:   end if
5:   if one string is empty then
6:     return  $\max\{|x|, |y|\}$ 
7:   end if
8:   if  $|x| = 1$  then
9:     Scan  $y$  and return  $|y| - \mathbf{1}_{x_1 \text{ occurs in } y}$ 
10:  end if
11:   $N \leftarrow |x| + |y|$ 
12:   $H \leftarrow$  the least power of two at least  $\lceil \log_2(N + 2) \rceil$ 
13:   $\ell \leftarrow$  the least power of two at least  $\max\{1, \log_2 H\}$ 
14:   $e \leftarrow \text{INSPECTACCURACY}(\varepsilon, H^2 + c_{\text{enc}})$ 
15:  if  $e = \text{INCOMPLETE}$  or  $\ell < 2^{1000}$  then
16:    return EXACTDP( $x, y$ )
17:  end if
18:  Write  $e = (e_{\text{num}}, e_{\text{den}})$ 
19:  if  $\ell e_{\text{num}} < 10e_{\text{den}}$  then
20:    return EXACTDP( $x, y$ )
21:  end if
22:  INITIALIZEPARAMETERS( $x, y, N, H, \ell$ )
23:   $U \leftarrow U_0$ , an initial-seed handle;  $A \leftarrow A_{\text{init}}$ ;  $s \leftarrow 0$ 
24:  while  $A > F$  do
25:     $s \leftarrow s + 1$ ;  $a \leftarrow A/F$ 
26:    for  $j = 1, \dots, H^2$  do
27:      Create immutable  $C_{s,j} = (\text{pass} = s, \text{copy} = j, U, A, a)$ 
28:    end for
29:    Create seed handle  $U_s$  with copies  $(C_{s,j})_{j=1}^{H^2}$ 
30:     $U \leftarrow U_s$ ;  $A \leftarrow 4a$ 
31:  end while
32:  Create immutable  $C_* = (\text{pass} = s + 1, \text{copy} = 0, U, A, a = 1)$ 
33:   $D' \leftarrow \text{READ}(\text{D}(C_*, S), I_{\text{root}}, (0, |y|))$ 
34:  return  $\lfloor \min\{N, \max\{0, D'\}\} \rfloor$ 
35: end function

```

9 Finite arithmetic and numerical work

The tables use integer and rational arithmetic. We first bound their representations on every execution, including executions with inaccurate seeds. We then apply the finite optimizer from Lemma 5.3 to the actual band lists.

9.1 Finite local calculations on inaccurate seeds

We first bound the numbers manipulated by the implementation. These bounds concern its finite local rules, rather than the global cone identities that were proved under a good seed.

Lemma 9.1. *On every execution, all local searches are finite. The warmup values are nonnegative integers bounded by $(2L_0 + 1)N$, and every refinement value satisfies $0 \leq d_t(I, q) \leq w_0(I, q)$. Put $Q = \ell^{10} + 1$, and let $h(I)$ be the remaining height of the fixed tree below I . Every returned $d_t(I, q)$ has denominator dividing $Q^{h(I)}$. The rational data used in band tests, group comparisons, and the numerical optimizer have bit length $H^{O(p_*+1)}$.*

Proof. Every seed entry is an integer in $[0, N]$ by clamping. In the local calculation of $w_0(I, q)$, a zero tested seed returns zero. Otherwise the scale range is finite, and the search uses grid endpoints at sup distance at most $U(I, q) \leq N$ from q . A cone height $\tilde{U}(I, r)$ is at most N , and its added charge is at most $2L_0N$. The default value when the candidate list is empty is N . Leaf values satisfy the same bound. Hence

$$0 \leq w_0(I, q) \leq (2L_0 + 1)N.$$

All grid endpoints are integers: their spacing is a power of two truncated below at one. Thus cone and connection charges are integers. Every Bellman minimum contains its local w_0 value and has nonnegative candidates. It follows that every w_j is a nonnegative integer at most w_0 at the same state. The average m has denominator T .

The value Π_t contains the Bellman option, so it too is at most w_0 . The clipped definition of d_t is nonnegative and at most Π_t , irrespective of seed accuracy. An empty online minimum is represented by the symbol $+\infty$; the formula then returns Π_t . No unbounded search or approximate numerical sentinel is needed for an empty list.

For the denominator claim, induct on physical tree height, for all indices simultaneously. Leaves return integers. Each actual candidate at a nonleaf is an integer charge plus a sum of child values, or an integer warmup value. All child denominators divide $Q^{h(I)-1}$. Minima and maxima select one of their arguments. The only further denominator operation in a returned value is division by $1 + \delta = Q/\ell^{10}$, which adds at most one power of Q . Sampling weights and numerical mixtures select actions; they are not themselves averaged into the returned table value. This proves the claim. In particular the table-value bit length is

$$O(H + J \log \ell).$$

The preliminary scale labels h_i are dyadic ceilings of positive rational quantities of bounded bit length. In every retained band, the explicit guard gives

$$b/M \leq h_i \leq h_\Sigma \leq 64Fb.$$

Failed guards simply discard bands. All prefix and wide minima use already bounded warmup values and integer charges. Tests for positivity of Z , the ratio $z(I, r)/Z$, band membership, and rounded endpoint validity are exact rational tests of polynomial bit length. A nonpositive or infinite Z is discarded before division.

A group comparison may use, in addition to a common power of Q , the product of the $O(\ell)$ integers $k_h \leq M$ as a denominator. This still has polynomially many bits. These sampling denominators are discarded when the chosen action is actually evaluated. Online coefficients are finite sums of earlier table values divided by dyadic h_i and multiplied by the displayed parameters. Their bit length is bounded by $H^{O(p_*+1)}$. On inaccurate seeds their magnitudes need not have the good-seed normalization, but this does not increase their bit length beyond that bound. The

first step $\xi_0 = 1$ in Lemma 5.3 makes its gap estimate independent of the initial linear objective magnitude. $\square$

The locality bounds themselves are also unconditional. The scale range for w_0 has ratio polynomial in A , and radius divided by grid spacing $O(A^2P)$. For the other local searches, the ratio of the endpoints of the scale interval is $128aF$. It admits $O((p_* + 1)\ell)$ scales. At any such scale, $w_0(I, q)/b \leq 16F$, so the square specified in the local rule meets

$$O((1 + F/\theta)^2)$$

cells. A cell has $O((1 + P)^2)$ representative candidates. All these lists can be generated by ranges of integer multiple indices, without scanning unrelated string positions.

Corollary 9.2. *For a band in the edit-distance algorithm, the numerical mixture in Section 5 can be constructed and sampled with local expected bit cost $2^{o(H)}$, using only already gathered child values of earlier indices. The storage used by this calculation is $2^{o(H)}$ words.*

Proof. At a wide-action endpoint there are $O(P + 1)$ possible grid positions. Thus the full wide list has at most $(C(P + 1))^{2M}$ members, for an absolute constant C , and the band is a sublist. Also $d_c \leq M(21P)^2$. With $H_{\text{opt}} = H$, this gives $G_0 = H^{O(p_*+1)}$. For these vertices, use the common denominator Mh_Σ : every Mh_i is an integer because the dyadic scales satisfy $h_i \geq b/M$ and $b \geq 1$. The guard $h_\Sigma \leq 64Fb$ bounds this denominator's bit length by $O(H)$, without multiplying denominators across the action list. Lemma 5.3 gives a polynomial number of bit operations in these quantities and in the bit length of each input rational. Lemma 9.1 makes the last quantity $H^{O(p_*+1)}$ too. The total is at most

$$2^{O(M(p_*+1)\ell)} H^{O(p_*+1)} = 2^{o(H)},$$

because $M \leq H^{1/20}$, $\ell = \Theta(\log H)$, and $p_* + 1 = O((\log \ell)^{1/4} + 1)$. The same bounds apply to storage, with a reused rejection buffer. Every list score and every numerical iteration uses the stored child inputs, rather than a new recursive calculation for each action. $\square$

10 Lazy evaluation and the expected work bound

The tables defined above need not be filled. We evaluate only entries requested by the final root query and store each completed result. Two different costs must be controlled. A wide search may inspect many actions using previously gathered child values. At the current index, however, recursive calls are restricted to the shared group samples, the chosen rounded actions, and the actual online draws. The bounds on recursive requests hold on every execution of the local rules, including executions with inaccurate seeds or unsuccessful group samples. We charge the expected numerical and sampling costs conditional on those requests.

10.1 Evaluation order and stored randomness

Consider an uncached request for $d_t(I, q)$. The local cone tests and warmup data determine the representatives, bands and rounded actions that it can use. Evaluating these tests may itself discover further warmup or seed requests. Once the bands are known, the wide Bellman search and the online optimizer gather the needed earlier-index child values and reuse them throughout their action enumerations. The group samples, selected rounded actions and sampled online actions also request current-index child values. The entry is then obtained by the exact minimum and maximum

in Definition 5.5 and cached. The distinction between gathering inputs and repeatedly using them is what permits large local action lists with few recursive calls.

These instructions are well founded before any accuracy event is considered. Number refinement passes in their order of construction, give w_j index j and d_t index $T + t$, and identify $d_0 = w_T$. Treat a seed median as its H^2 preceding-pass refinement requests. Inline averages, bands, groups, and online choices into their requesting table entries. Every table dependency strictly decreases the lexicographic triple

$$(\text{pass}, \text{index}, \text{remaining tree height}).$$

A current-index request descends to a child. Warmup and history requests decrease the index; seed requests enter an earlier pass, where the index may reset. Initial entries terminate this order because their estimators do not query any other initial entry. Different copy labels never reset the pass or index. Repeated and empty intervals still have distinct node labels and smaller remaining height at their children.

Table 2 lists the complete cache keys. The initial entry’s private stream supplies all its guesses and thresholds. Group streams also distinguish the scale label h and sample number.

Stored object	Complete key
Initial seed entry	Node label and endpoint pair
Later seed entry	Pass, node label, and endpoint pair
Warmup or refinement entry	Table kind, pass, copy, node label, index, and endpoint pair
Warmup average	Pass, copy, node label, and endpoint pair
Band data	Pass, copy, node label, scale b , and representative r
Group selection and actual cost	Pass, copy, node label, scale b , two cell indices, and t
Online action	Pass, copy, node label, requesting pair q , scale b , and t

Table 2: Keys for memoized values and random choices. Band data do not depend on t . A group key does not include the requesting pair q . The two table names identify $d_0 = w_T$.

In particular, a requested group is computed over its entire specified cell. Its result does not depend on which query first requested it. Repeated list processing and numerical iterations use dictionary lookups for previously gathered recursive inputs. Each lookup is charged.

Generate fresh fair random bits on the first calculation of each random object, and retain its outcome. Different copies, passes, nodes, and specified random objects have independent sources. This can be viewed equivalently as assigning independent bit streams to all keys in advance. By the dependency order above, deciding to request an object uses only already exposed streams. This equivalence is a distributional description, not an instruction to allocate all streams or tables.

For a given refinement copy, the complete child tables depend on descendant streams and earlier passes, but not on fresh streams of the current parent. Conditioning on those tables consequently leaves its group samples and online draws with their specified independent laws. This justifies the conditioning used in the accuracy analysis. The whole-domain seed guarantees are retained; adaptively requesting a seed entry does not replace such a guarantee by one for a fixed query.

For the expected running time, we use a different conditioning. Before the first request of an initial entry, none of its private random bits has been exposed. Conditional on the previous execution and its first request, the expected cost bound of Theorem 3.5 still applies. Later requests only look up its stored answer. This argument does not condition on the initial entry being correct.

The dispatch below sends each complete key to its defining procedure. It resolves the $d_0 = w_T$ alias before looking up the key. For a random key k , ω_k is created on its first evaluation and is used only by that owner. Only initial entries, group selections, and online choices draw random bits.

Algorithm 24 Table reads and complete cache dispatch

Input: A table descriptor for READ, or a key of one of the listed types for GET

```

1: function READ( $R, I, q$ )
2:   return GET( $(R, I, q)$ )
3: end function
4: function GET( $k$ )
5:   if  $k = (D(C, 0), I, q)$  then
6:      $k \leftarrow (W(C, T), I, q)$ 
7:   end if
8:   if  $k \in \mathcal{D}$  then
9:     return  $\mathcal{D}[k]$ 
10:  end if
11:  if  $k = (U_0, I, q)$  then
12:     $v \leftarrow \text{INITIALENTY}(I, q, \omega_k)$ 
13:  else if  $k = (U_s, I, q)$  with  $s > 0$  then
14:     $v \leftarrow \text{SEEDENTRY}(U_s, I, q)$ 
15:  else if  $k = (W(C, j), I, q)$  then
16:     $v \leftarrow \text{WARMUP}(C, I, q, j)$ 
17:  else if  $k = (M(C), I, q)$  then
18:     $v \leftarrow \text{MEAN}(C, I, q)$ 
19:  else if  $k = (D(C, t), I, q)$  with  $t > 0$  then
20:     $v \leftarrow \text{REFINEMENTENTRY}(C, I, q, t)$ 
21:  else if  $k = (\text{Band}, C, I, b, r)$  then
22:     $v \leftarrow \text{BANDDATA}(C, I, b, r)$ 
23:  else if  $k = (\text{Group}, C, I, b, c_{\text{cell}}, t)$  then
24:     $v \leftarrow \text{GROUPCHOICE}(C, I, b, c_{\text{cell}}, t, \omega_k)$ 
25:  else if  $k = (\text{Online}, C, I, q, b, t)$  then
26:     $v \leftarrow \text{ONLINECHOICE}(C, I, q, b, t, \omega_k)$ 
27:  end if
28:   $\mathcal{D}[k] \leftarrow v$ 
29:  return  $v$ 
30: end function

```

10.2 The current-index child budget

The following count is the reason for using rounded actions and samples shared across a cell. At a small scale h , many rounded states can occur at one child, but few children are sampled. Their respective factors b/h and h/b cancel. At a large scale, the rounded list is already small enough to inspect every child. The count concerns distinct recursive inputs; repeated uses of an input are charged later as lookups.

Lemma 10.1. *Evaluating one refinement entry $d_t(I, q)$ requires at most $M\ell^{O(1)}$ distinct child entries at the same pass, copy, and index t . This bound holds for all seeds and all sample outcomes.*

Proof. Fix a group at scale b , a scale label h , and a child i . Put $D_h = \max(1, \theta h)$. Across the whole cell, all queried rounded states with $h_i = h$ use this common endpoint spacing. The wide action guard puts their first endpoints in an interval of length $O(b)$: r^- varies by at most θb within the cell, and the shift radius is at most $10b$. Once the first endpoint is fixed, the guard $|z(I_i, s_i)| \leq 2h$

restricts the second endpoint to an interval of length $4h$. The number of distinct states is at most

$$C(1 + b/D_h)(1 + h/D_h) \leq C'\theta^{-2}(1 + b/h),$$

for absolute constants C, C' . This estimate also holds when $\theta h < 1$, using $D_h \geq \theta h$. Requiring legitimate ordered endpoints in $[0, |y|]$ can only reduce the count.

There are two cases in the group calculation. If $k_h < M$, then $h \geq b/M$ and $Q_1 \geq 1$ imply

$$k_h \leq 2Q_1 M h / b, \quad h/b < 1/Q_1.$$

At most k_h distinct child labels occur in the sample. Multiplying the rounded-state count by this number gives at most

$$C'\theta^{-2}k_h(1 + b/h) \leq C''M\theta^{-2}Q_1$$

distinct inputs. If $k_h = M$, the equality $\min(M, \lceil Q_1 M h / b \rceil) = M$ implies $Q_1 M h / b > M - 1$. Since $M \geq 2$ in this branch, $b/h < 2Q_1$. Evaluating the exact part for all children consequently uses at most

$$C'M\theta^{-2}(1 + 2Q_1)$$

distinct inputs as well. This proves $M\ell^{O(1)}$ for one group and one scale label, without an accuracy event.

There are $O(\log(FM)) = O(\ell)$ scale labels. The number of b scales and cells requested by one query is polynomial in ℓ , as established above. Once a winning rounded action is chosen, evaluating its actual cost adds at most M child calls per group. Finally the online rule draws just one action for each applicable b scale; evaluating that actual action adds at most M child calls per scale. These factors preserve the asserted bound. $\square$

All other recursive inputs are earlier in the evaluation order. Constructing w_0 uses a previous seed. Computing bands, prefix minima, and scale labels uses warmup data. In Π_t the wide Bellman option uses d_{t-1} on children. The online coefficient vector uses d_j only for $j < t$; it does not require the past transcript of actions at the parent. The numerical procedure of Section 9 introduces no hidden current-index child queries.

10.3 Earlier inputs and local arithmetic

Lemma 10.2. *After collapsing the finite auxiliary calculations into table-entry evaluations, one warmup or refinement entry has at most $H^{O(p_*+1)}$ distinct inputs at earlier indices or passes. Apart from its recursive inputs, its expected bit cost is $2^{o(H)}$, uniformly conditional on the data exposed before its fresh random draws. Its local storage is at most $2^{o(H)}$ on every execution.*

Proof. For one representative, each endpoint in a wide action has $O(P+1)$ positions, since $b/g_b \leq P$. A child has $O((P+1)^2)$ states in the wide list. Multiplying by the number of children, representative candidates, cells, scales, and history indices still gives $H^{O(p_*+1)}$ distinct recursive inputs. There are at most $T + S + 1 = O(H)$ history indices. The searches for w_0 have the same form of bound. An earlier seed query expands to H^2 copies of the previous refinement when required; this factor fits the same estimate.

Gather and store these distinct child inputs before repeatedly scoring action lists. The wide list has at most $(C(P+1))^{2M}$ actions. For each internal cut, a prefix minimum enumerates the independently allowed states of its first children. The algorithm evaluates it only at the grid shifts being tested for band membership; the real argument in the definition of Φ_j does not require a search over a continuum. These prefix lists and the band candidates have the same exponential

bound. Scanning them at all candidate cuts and shifts multiplies the list size by at most a fixed power and by polynomial factors. Direct enumeration, including all guard tests, has local arithmetic count

$$2^{O(M(p_*+1)\ell)} H^{O(p_*+1)}.$$

Lemmas 9.1 and 5.3 give polynomial bit costs in the stated parameters for each arithmetic operation and for the numerical routine. Hence all local work and storage, apart from exact rejection-sampling time, are $2^{o(H)}$. Rejection sampling has that bound in expectation, with a fixed buffer. Uniform child indices can likewise be sampled by charged fair bits; here M is a power of two.

Use a balanced search dictionary for gathered and globally stored inputs. The keys consist of the bounded integer labels specified above, and their comparisons use polynomially many bits in H . The number of possible keys is polynomial in N : there are $O(MN)$ node labels, at most $(N+1)^2$ states per node, polynomially many pass/copy/index choices, and polynomially bounded cell-index ranges since $\theta b \geq \theta$. Thus each dictionary operation has $H^{O(1)}$ bit cost. Repeated lookups while scoring lists or optimizing are included in the displayed local work, not counted as new recursive inputs. $\square$

10.4 Counting the dependency paths

We now quantify the dependency order from the first subsection. After inlining seed medians and auxiliary calculations, each dependency has one of two forms. A same-pass, same-index dependency descends to a child; Lemma 10.1 bounds its number by

$$D_{\text{same}} = M\ell^{C_1}.$$

Every other dependency either decreases the index in the same pass or switches to an earlier pass, and its number is bounded by

$$D_{\text{earlier}} = H^{C_2(p_*+1)}$$

for absolute constants C_1, C_2 . Such dependencies stay at the same physical node or descend; they never ascend the tree. Auxiliary band and group calculations have been collapsed into these evaluations, so they create no same-index cycle at a node.

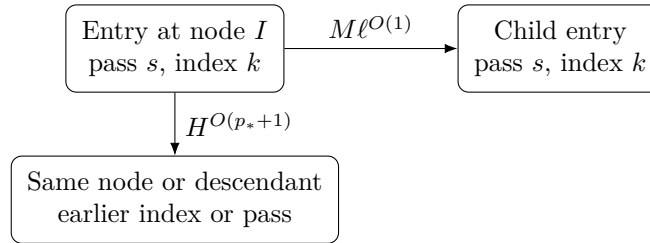


Figure 4: The two recursive input budgets. Wide action enumeration and numerical iterations reuse the gathered inputs and contribute local work. Only the horizontal edge keeps both pass and index unchanged.

Lemma 10.3. *The number of expanded table evaluations at source depth j is at most $M^j 2^{o(H)}$, uniformly for $0 \leq j \leq J$, on every execution.*

Proof. Let s_{pass} be the number of refinement passes, including the final pass, and put

$$K_{\text{dep}} = s_{\text{pass}}(T + S + 1).$$

A dependency path ending at depth j has at most j same-index edges, because each descends one level. It has at most K_{dep} other edges: indices strictly decrease between pass switches, and passes strictly decrease at switches. Copy identities do not reset this order.

Unfolding the stored dependency graph only overcounts calculations. The number of paths ending at depth j is at most

$$\begin{aligned} \sum_{r=0}^j \sum_{s=0}^{K_{\text{dep}}} \binom{r+s}{r} D_{\text{same}}^r D_{\text{earlier}}^s \\ \leq (j+1)(K_{\text{dep}}+1)2^{j+K_{\text{dep}}} M^j \ell^{C_1 j} H^{C_2(p_*+1)K_{\text{dep}}}. \end{aligned}$$

Some index-decreasing edges descend too; ignoring that additional descent only enlarges this bound.

Write $q_* = p_* + 1$. Lemma 2.2 and Proposition 8.3 give

$$J = O(H/\ell), \quad T = O(H^{3/4}) = o(S), \quad K_{\text{dep}} = O\left(\frac{Hq_*}{\ell \log \ell}\right).$$

Consequently

$$\begin{aligned} J \log \ell &= O(H \log \ell / \ell) = o(H), \\ q_* K_{\text{dep}} \log H &= O\left(\frac{Hq_*^2}{\log \ell}\right) = o(H), \\ J + K_{\text{dep}} &= o(H). \end{aligned}$$

For the second limit, use $q_*^2 = O(\sqrt{\log \ell} + 1)$. All factors beyond M^j in the dependency-path bound are $2^{o(H)}$, uniformly for $0 \leq j \leq J$. This proves the claimed bound. $\square$

10.5 Expected time and logarithmic-word storage

Theorem 10.4. *The large-input algorithm has worst-case expected running time $N^{1+o(1)}$ on the logarithmic-word RAM, over its internal randomness. This includes input processing, preprocessing, initial and refined table calculations, all rational arithmetic, sampling, and data-structure operations. Its workspace on every execution is bounded by a fixed polynomial in N . Together with the exact fallbacks, the same program has the claimed expected asymptotic bound for each fixed rational $\varepsilon \in (0, 1)$.*

Proof. Let $n_x = |x|$. At depth j , an interval has width at most $\lceil n_x / M^j \rceil$. Theorem 3.5 bounds the expected cost of a first initial-entry request there by

$$\left(1 + \lceil n_x / M^j \rceil\right) 2^{o(H)}.$$

The conditional first-request argument above makes this valid after any preceding adaptive execution. Equivalently, enumerate the possible request slots in an unfolded computation: at each slot that requests a new initial entry, condition on the past and use its independent fresh stream. Lemma 10.3 bounds the number of request slots, so these conditional expectations can be summed. Repeat requests have only the charged dictionary cost. The same conditional expectation argument applies to each exact mixture sampler, whose expected local cost is uniformly bounded.

Combining Lemma 10.2 with Lemma 10.3, all these costs are at most

$$\begin{aligned}
2^{o(H)} \sum_{j=0}^J M^j \left(1 + \lceil n_x / M^j \rceil\right) &\leq 2^{o(H)} \sum_{j=0}^J (2M^j + n_x) \\
&\leq 2^{o(H)} (O(M^J) + (J+1)n_x) \\
&\leq (n_x + 1)2^{o(H)}.
\end{aligned}$$

Here $M^J \leq M \max(1, n_x)$ and $M, J = 2^{o(H)}$. No seed or group correctness event was used. Sorting the occurrence data and the other input work cost $O(N \log(N+2))$, and leaf queries cost $O(\log(N+2))$ each. Since $H = \Theta(\log(N+2))$, the complete expected bound is $N^{1+o(1)}$.

For storage, the deterministic dependency count bounds the number of stored table entries by $N^{1+o(1)}$. Each such evaluation creates at most $H^{O(p_*+1)}$ auxiliary representative, group and draw records, so their total satisfies the same bound. Their keys and scalar fields have the bit bounds of Lemma 9.1. Each local action list and numerical computation has at most $2^{o(H)} = N^{o(1)}$ scratch size by Lemma 10.2; even retaining such lists or mixtures with their auxiliary records keeps the total refinement storage at $N^{1+o(1)}$. A fully expanded initial estimator also has polynomial worst-case space: its tree has $O(Bn)$ nodes on padded length n , its shift arrays have length $O(n)$, and its numbers have polynomial bit length in H and B . Pruned arrays are represented by markers. Its scratch can be released after storing the completed integer entry. Retaining the scratch of nested calls still gives a polynomial bound, since dependency depth is at most $J + K_{\text{dep}}$ plus the initial-tree depth. Rejection loops reuse their buffers and do not enlarge storage when they run for additional trials.

Thus there is an absolute polynomial space bound in N in the large branch. Every workspace address fits a constant number of $O(\log(N+2))$ -bit words. Larger rational numbers are represented in multiple such words and processed by charged ordinary integer algorithms, as used above. Input symbols and the padding tag also fit a constant number of these words, with no constant-alphabet assumption. The finite parameter-inspection rule and exact dynamic-programming fallback of Section 2 use polynomially addressed memory as well.

Finally, for each fixed rational accuracy encoding, its length is eventually below the inspection limit, and the two remaining fallback conditions also cease for sufficiently large N . Their sizes are bounded as a function of that fixed parameter. They do not change the asserted asymptotic bound, although they impose no efficiency claim uniform in a varying ε . Together with the success probability established in Section 8, this proves Theorem 1.1. $\square$

References

- [1] Alexandr Andoni and Negev Shekel Nosatzki. Edit distance in near-linear time: It’s a constant factor. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*, pages 990–1001, 2020. doi: 10.1109/FOCS46700.2020.00096. Full version: arXiv:2005.07678v2, 2022.
- [2] Alexandr Andoni and Krzysztof Onak. Approximating edit distance in near-linear time. *SIAM Journal on Computing*, 41(6):1635–1648, 2012. doi: 10.1137/090767182. Preliminary version in STOC 2009, pages 199–204.
- [3] Alexandr Andoni, Robert Krauthgamer, and Krzysztof Onak. Polylogarithmic approximation for edit distance and the asymmetric query complexity. In *2010 IEEE 51st Annual Symposium*

- on *Foundations of Computer Science*, pages 377–386. IEEE Computer Society, 2010. doi: 10.1109/FOCS.2010.43. URL <https://arxiv.org/abs/1005.4033v1>. Theorem numbers refer to the full version, arXiv:1005.4033v1.
- [4] Alexandr Andoni, Robert Krauthgamer, and Krzysztof Onak. Streaming algorithms via precision sampling. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 363–372. IEEE Computer Society, 2011. doi: 10.1109/FOCS.2011.82. URL <https://arxiv.org/abs/1011.1263v2>. Full version titled “Streaming Algorithms from Precision Sampling”; Lemma 1.2 in arXiv:1011.1263v2.
 - [5] Arturs Backurs and Piotr Indyk. Edit distance cannot be computed in strongly subquadratic time (unless SETH is false). *SIAM Journal on Computing*, 47(3):1087–1097, 2018. doi: 10.1137/15M1053128.
 - [6] Ziv Bar-Yossef, T. S. Jayram, Robert Krauthgamer, and Ravi Kumar. Approximating edit distance efficiently. In *Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science*, pages 550–559, 2004. doi: 10.1109/FOCS.2004.14.
 - [7] Tuğkan Batu, Funda Ergün, Joe Kilian, Avner Magen, Sofya Raskhodnikova, Ronitt Rubinfeld, and Rahul Sami. A sublinear algorithm for weakly approximating edit distance. In *Proceedings of the 35th Annual ACM Symposium on Theory of Computing*, pages 316–324, 2003. doi: 10.1145/780542.780590.
 - [8] Tuğkan Batu, Funda Ergün, and Cenk Sahinalp. Oblivious string embeddings and edit distance approximations. In *Proceedings of the 17th Annual ACM–SIAM Symposium on Discrete Algorithms*, pages 792–801, 2006. doi: 10.1145/1109557.1109644.
 - [9] Joshua Brakensiek and Aviad Rubinfeld. Constant-factor approximation of near-linear edit distance in near-linear time. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 685–698, 2020. doi: 10.1145/3357713.3384282.
 - [10] Diptarka Chakraborty, Debarati Das, Elazar Goldenberg, Michal Koucký, and Michael Saks. Approximating edit distance within constant factor in truly sub-quadratic time. *Journal of the ACM*, 67(6):36:1–36:22, 2020. doi: 10.1145/3422823.
 - [11] Debarati Das, Evangelos Kipouridis, and Tomasz Kociumaka. Metric weighted edit distance: A $(3 + \varepsilon)$ -approximation in $\tilde{O}_\varepsilon(n^{1.6})$ time, 2026. URL <https://arxiv.org/abs/2609.20796v1>. Version 1, September 17, 2026.
 - [12] Marguerite Frank and Philip Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, 3(1–2):95–110, 1956. doi: 10.1002/nav.3800030109. URL <https://doi.org/10.1002/nav.3800030109>.
 - [13] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. doi: 10.1006/jcss.1997.1504. URL <https://doi.org/10.1006/jcss.1997.1504>.
 - [14] Piotr Indyk and David Woodruff. Optimal approximations of the frequency moments of data streams. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*. Association for Computing Machinery, 2005. URL <https://www.cs.cmu.edu/afs/cs/user/dwoodruf/www/iw05.pdf>.

- [15] Martin Jaggi. Revisiting Frank–Wolfe: Projection-free sparse convex optimization. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 427–435. PMLR, 2013. URL <https://proceedings.mlr.press/v28/jaggi13.html>.
- [16] Michal Koucký and Michael Saks. Constant factor approximations to edit distance on far input pairs in nearly linear time. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 699–712, 2020. doi: 10.1145/3357713.3384307.
- [17] Gad M. Landau and Uzi Vishkin. Fast string matching with k differences. *Journal of Computer and System Sciences*, 37(1):63–78, 1988. doi: 10.1016/0022-0000(88)90045-1.
- [18] Gad M. Landau and Uzi Vishkin. Fast parallel and serial approximate string matching. *Journal of Algorithms*, 10(2):157–169, 1989. doi: 10.1016/0196-6774(89)90010-2.
- [19] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966. English translation of Doklady Akademii Nauk SSSR 163(4), 845–848 (1965).
- [20] Ethan Mader, Borna Tavasoli, and Jihan Wang. A strongly subquadratic $(3 + \varepsilon)$ -approximation for weighted edit distance over arbitrary metrics, 2026. URL <https://arxiv.org/abs/2609.14873v1>. Version 1, September 14, 2026.
- [21] Xiao Mao and Aviad Rubinfeld. Approximation schemes for edit distance and LCS in quasi-strongly subquadratic time. In *Proceedings of the 58th Annual ACM Symposium on Theory of Computing*. Association for Computing Machinery, 2026. doi: 10.1145/3798129.3800789. URL <https://arxiv.org/abs/2603.29702v1>. Theorem numbers refer to the full version, arXiv:2603.29702v1.
- [22] William J. Masek and Michael S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20(1):18–31, 1980. doi: 10.1016/0022-0000(80)90002-1.
- [23] Eugene W. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1:251–266, 1986. doi: 10.1007/BF01840446.
- [24] OpenAI. Edit Distance in ℓ_1 : Matching Bounds up to Constants in the Exponent. OpenAI Math Release preprint [OAI:Edit-Distance-in-l1-Matching-Bounds-up-to-Constants-in-the-Exponent-September-27-2026](https://arxiv.org/abs/2609.14873v1), 2026.
- [25] Rafail Ostrovsky and Yuval Rabani. Low distortion embeddings for edit distance. *Journal of the ACM*, 54(5):23:1–23:16, 2007. doi: 10.1145/1284320.1284322.
- [26] Alexander Rakhlin and Karthik Sridharan. Online learning with predictable sequences. In *Proceedings of the 26th Annual Conference on Learning Theory*, volume 30 of *Proceedings of Machine Learning Research*, pages 993–1019. PMLR, 2013. URL <https://proceedings.mlr.press/v30/Rakhlin13.html>.
- [27] Joel A. Tropp. ACM 217: Probability in high dimensions. Caltech CMS Lecture Notes 2021-01, California Institute of Technology, Pasadena, March 2021. URL <https://tropp.caltech.edu/notes/Tro21-Probability-High-LN-corr.pdf>. Corrected March 2023; Theorem 4.16.

- [28] Esko Ukkonen. Algorithms for approximate string matching. *Information and Control*, 64(1–3): 100–118, 1985. doi: 10.1016/S0019-9958(85)80046-2.
- [29] Robert A. Wagner and Michael J. Fischer. The string-to-string correction problem. *Journal of the ACM*, 21(1):168–173, 1974. doi: 10.1145/321796.321811.